

Cadence SKILL Development Reference

Product Version 6.1.6
August 2014

© 1990–2013 Cadence Design Systems, Inc. All rights reserved.
Printed in the United States of America.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Open SystemC, Open SystemC Initiative, OSCI, SystemC, and SystemC Initiative are trademarks or registered trademarks of Open SystemC Initiative, Inc. in the United States and other countries and are used with permission.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence's trademarks, contact the corporate legal department at the address shown above or call 800.862.4522. All other trademarks are the property of their respective holders.

Restricted Permission: This publication is protected by copyright law and international treaties and contains trade secrets and proprietary information owned by Cadence. Unauthorized reproduction or distribution of this publication, or any portion of it, may result in civil and criminal penalties. Except as specified in this permission statement, this publication may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, without prior written permission from Cadence. Unless otherwise agreed to by Cadence in writing, this statement grants Cadence customers permission to print one (1) hard copy of this publication subject to the following conditions:

1. The publication may be used only in accordance with a written agreement between Cadence and its customer.
2. The publication may not be modified in any way.
3. Any authorized copy of the publication or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement.
4. The information contained in this document cannot be used in the development of like products or software, whether for internal or external use, and shall not be used for the benefit of any other party, whether or not for consideration.

Disclaimer: Information in this publication is subject to change without notice and does not represent a commitment on the part of Cadence. Except as may be explicitly set forth in such agreement, Cadence does not make, and expressly disclaims, any representations or warranties as to the completeness, accuracy or usefulness of the information contained in this document. Cadence does not warrant that use of such information will not infringe any third party rights, nor does Cadence assume any liability for damages or costs of any kind that may result from use of such information.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227-14 and DFAR252.227-7013 et seq. or its successor

Contents

<u>Preface</u>	7
<u>Scope of this Manual</u>	7
<u>Licensing in Cadence SKILL Language</u>	7
<u>About the SKILL Language</u>	8
<u>Related Documents</u>	8
<u>Installation, Environment, and Infrastructure</u>	8
<u>Section Names and Meanings</u>	9
<u>Typographic and Syntax Conventions</u>	9
<u>SKILL Syntax Examples</u>	11
<u>Identifiers Used to Denote Data Types</u>	12
<u>Additional Learning Resources</u>	13
<u>How to Contact Technical Support</u>	14
<u>1</u>	
<u>Overview</u>	15
<u>Cadence SKILL Development Tools</u>	16
<u>Quick Reference Tool: Finder</u>	17
<u>Copying and Pasting Code Examples</u>	17
<u>2</u>	
<u>Profiler Functions</u>	19
<u>profile</u>	19
<u>profileReset</u>	21
<u>profileSummary</u>	22
<u>unprofile</u>	25
<u>3</u>	
<u>Lint Functions</u>	27
<u>skDisableMessage</u>	27
<u>skDisableMessageBlock</u>	29

Cadence SKILL Development Reference

<u>skEnableMessageBlock</u>	31
<u>skIgnoreMessage</u>	33
<u>sklint</u>	35
<u>skUnignoreMessage</u>	40

4

Context Functions 43

<u>callInitProc</u>	43
<u>callUserAutoInitProc</u>	45
<u>checkContextBit</u>	46
<u>defCapDepends</u>	47
<u>defCapPrefixes</u>	48
<u>defInitProc</u>	50
<u>isContextLoaded</u>	51
<u>loadContext</u>	53
<u>loadTopContextForms</u>	55
<u>saveContext</u>	57
<u>setContext</u>	59
<u>setSaveContextVersion</u>	60
<u>getCurSaveContextVersion</u>	61
<u>getNativeContextVersion</u>	62
<u>getCompatContextVersion</u>	63

5

Debug Functions 65

<u>break</u>	65
<u>breakpt</u>	66
<u>breakptMethod</u>	69
<u>clear</u>	71
<u>cont, continue</u>	72
<u>count</u>	73
<u>debugQuit</u>	74
<u>debugStatus</u>	75
<u>dump</u>	76
<u>gcsummary</u>	78

Cadence SKILL Development Reference

<u>getAllLoadedFiles</u>	80
<u>getCallingFunction</u>	81
<u>getFunctions</u>	83
<u>getGFbyClass</u>	84
<u>ilAddTopLevelErrorHandler</u>	85
<u>ilDebugCountLevels</u>	86
<u>ilGetGFbyClass</u>	87
<u>ilGetIdeSessionWindow</u>	88
<u>ilRemoveMethod</u>	89
<u>ilRemoveTopLevelErrorHandler</u>	90
<u>ilSlotBoundp</u>	91
<u>ilToolBox</u>	92
<u>inNext</u>	93
<u>inStepOut</u>	95
<u>installDebugger</u>	97
<u>listAlias</u>	98
<u>listFunctions</u>	99
<u>listVariables</u>	101
<u>memoryAllocated</u>	102
<u>next</u>	103
<u>pp</u>	105
<u>printFunctions</u>	106
<u>printObject</u>	108
<u>printstruct</u>	110
<u>printVariables</u>	111
<u>removeMethod</u>	112
<u>resume</u>	113
<u>skillDebugger</u>	114
<u>skillDevStatus</u>	116
<u>stacktrace</u>	117
<u>step</u>	119
<u>stepend</u>	121
<u>stepout</u>	123
<u>toplevel</u>	125
<u>tracef</u>	127
<u>tracelevlimit</u>	130

Cadence SKILL Development Reference

<u>tracelevunlimit</u>	132
<u>tracep</u>	133
<u>tracev</u>	134
<u>unbreakpt</u>	135
<u>unbreakptMethod</u>	137
<u>uncount</u>	138
<u>uninstallDebugger</u>	140
<u>untrace</u>	141
<u>untracep</u>	142
<u>untracev</u>	143
<u>unwatch</u>	144
<u>watch</u>	145
<u>where</u>	146
<u>whereIs</u>	149

6

<u>Finder Functions</u>	151
<u>startFinder</u>	151

7

<u>Tabulator Functions</u>	153
<u>skTabulate</u>	153
<u>skTabulateSKILL</u>	156

8

<u>SKILL IDE Functions</u>	157
<u>ilgInvokeIDE</u>	157
<u>ilgRunSKILLIDE</u>	158
<u>ilgLastDir</u>	159
<u>ilgSetLastDir</u>	160
<u>ilgAddRecentFiles</u>	161

Preface

This manual provides information on using the SKILL Development tools, which include the Lint, Profiler, Finder, Code Browser, Surveyor, and Debugger. These tools, especially the Finder, provides quick reference information for syntax and abstract statements for SKILL language functions and application programming interfaces (APIs). It also provides information on the basic SKILL language functions.

It is aimed at the following users:

- Programmers beginning to program in SKILL
- CAD developers who have experience programming in SKILL, both Cadence internal users and Cadence customers
- CAD integrators

Scope of this Manual

The SKILL functions described in this manual can be used in either IC6.1.6, ICADV12.1, or both of these releases. Functions that are supported only in a particular release are identified using the **(ICADV12.1 ONLY)** or **(IC6.1.6 ONLY)** text at the beginning of the function description. All other functions are supported in both releases.

Important

Only the functions and arguments described in this manual are available for public use. Any undocumented functions or arguments are likely to be private and could be subject to change without notice. It is recommended that you check with your Cadence representative before using them.

Licensing in Cadence SKILL Language

Product license 111 is checked out at `skill` executable startup or at workbench startup.

For information on licensing in the Cadence SKILL Language, see [*Virtuoso Software Licensing and Configuration Guide*](#).

About the SKILL Language

The SKILL programming language lets you customize and extend your design environment. SKILL provides a safe, high-level programming environment that automatically handles many traditional system programming operations, such as memory management. SKILL programs can be immediately executed in the Cadence environment.

SKILL is ideal for rapid prototyping. You can incrementally validate the steps of your algorithm before incorporating them in a larger program.

Storage management errors are persistently the most common reason cited for schedule delays in traditional software development. SKILL's automatic storage management relieves your program of the burden of explicit storage management. You gain control of your software development schedule.

SKILL also controls notoriously error-prone system programming tasks like list management and complex exception handling, allowing you to focus on the relevant details of your algorithm or user interface design. Your programs will be more maintainable because they will be more concise.

The Cadence environment allows SKILL program development such as user interface customization. The SKILL Development Environment contains powerful tracing, debugging, and profiling tools for more ambitious projects.

SKILL leverages your investment in Cadence technology because you can combine existing functionality and add new capabilities.

SKILL allows you to access and control all the components of your tool environment: the User Interface Management System, the Design Database, and the commands of any integrated design tool. You can even loosely couple proprietary design tools as separate processes with SKILL's interprocess communication facilities.

Related Documents

The following documents provide more information about SKILL and other topics discussed in this guide.

Installation, Environment, and Infrastructure

- For more information on installing Cadence products, see the [*Cadence Installation Guide*](#)

- For information on using the SKILL development tools, refer to *Cadence SKILL IDE User Guide*, which is available when you select the *Help* button on the SKILL IDE window.
- For information on database SKILL functions, including data access functions, see the *Virtuoso Design Environment SKILL Reference*. It contains APIs for the graphics editor, database access, design management, technology file administration, online environment, design flow, user entry, display lists, component description format, and graph browser.
- *Cadence User Interface SKILL Reference* contains APIs for management of windows and forms.
- *Virtuoso Software Licensing and Configuration Guide* contains SKILL licensing functions.

Section Names and Meanings

Each function described in this book can have up to seven sections. Not every section is required for every function description.

Syntax	The syntax requirements for this function.
Prerequisites	Steps required before calling this function.
Description	A brief phrase identifying the purpose of the function and the text description of the operation performed by the function.
Arguments	An explanation of the arguments input to the function.
Return Value	An explanation of the value returned by the function.
Example	Actual SKILL code using this function.
References	Other functions that are relevant to the operation of this function: ones with partial or similar functionality or which could be called by or could call this function. Sections in this manual which explain how to use this function.

Typographic and Syntax Conventions

The following typographic and syntax conventions are used in this document.

Cadence SKILL Development Reference

Preface

`literal (LITERAL)`

Nonitalic (UPPERCASE) words indicate keywords that you must enter literally. These keywords represent command (function, routine) or option names.

`argument (z_argument)`

Words in italics indicate text that you must replace with an appropriate argument. The prefix (in this case, *z_*) indicates the data type that the argument can accept. Names are case sensitive. Do not type the data type or underscore before your arguments.

|

Vertical bars (OR-bars) separate the choice of options. They take precedence over any other character.

[]

Brackets denote optional arguments. When used with vertical bars, they enclose a list of choices from which you can choose one.

{ }

Braces are used with vertical bars and enclose a list of choices from which you must choose one.

...

Three dots (. . .) indicate that you can repeat the previous argument. If you use them with brackets, you can specify zero or more arguments. If they are used without brackets, you must specify at least one argument, but you can specify more.

```
argument... ;specify at least one,  
            ;but more are possible
```

```
[argument]... ;specify zero or more
```

, ...

A comma and three dots together indicate that if you specify more than one argument, you must separate those arguments by commas.

=>

A right arrow points to the return values of the function. Variable values returned by the software are shown in italics. Returned literals, such as `t` and `nil`, are in plain text. The right arrow is also used in code examples in SKILL manuals.

/

Separates the possible values that can be returned by a Cadence SKILL language function.

Cadence SKILL Development Reference

Preface

`text` Indicates names of manuals, menu commands, form buttons, and form fields.

SKILL Syntax Examples

The following examples show typical syntax characters used in SKILL.

Example 1

```
list( g_arg1 [g_arg2] ...) => l_result
```

This example illustrates the following syntax characters.

<code>list</code>	Plain type indicates words that you must enter literally.
<code><i>g_arg1</i></code>	Words in italics indicate arguments for which you must substitute a name or a value.
<code>()</code>	Parentheses separate names of functions from their arguments.
<code>_</code>	An underscore separates an argument type (left) from an argument name (right).
<code>[]</code>	Brackets indicate that the enclosed argument is optional.
<code>...</code>	Three dots indicate that the preceding item can appear any number of times.
<code>=></code>	A right arrow points to the description of the return value of the function. Also used in code examples in SKILL manuals.
<code>l_result</code>	All SKILL functions compute a data value known as the return value of the function.

Example 2

```
needNCells( s_cellType | st_userType x_cellCount) => t / nil
```

This example illustrates two additional syntax characters.

<code> </code>	Vertical bars separate a choice of required options.
<code>/</code>	Slashes separate possible return values.

Identifiers Used to Denote Data Types

The Cadence SKILL language supports several data types to identify the type of value you can assign to an argument.

Data types are identified by a single letter followed by an underscore; for example, *t* is the data type in *t_viewNames*. Data types and the underscore are used as identifiers only; they should not be typed.

Prefix	Internal Name	Data Type
<i>a</i>	array	array
<i>A</i>	amsobject	AMS object
<i>b</i>	ddUserType	DDPI object
<i>B</i>	ddCatUserType	DDPI category object
<i>C</i>	opfcontext	OPF context
<i>d</i>	dbobject	Cadence database object (CDBA)
<i>e</i>	envobj	environment
<i>f</i>	flonum	floating-point number
<i>F</i>	opffile	OPF file ID
<i>g</i>	general	any data type
<i>G</i>	gdmSpecIIUserType	gdm spec
<i>h</i>	hdbobject	hierarchical database configuration object
<i>l</i>	list	linked list
<i>L</i>	tc	Technology file time stamp
<i>m</i>	nmplIUserType	nmplI user type
<i>M</i>	cdsEvalObject	—
<i>n</i>	number	integer or floating-point number
<i>o</i>	userType	user-defined type (other)
<i>p</i>	port	I/O port
<i>K</i>	mapioobject	MAPI object
<i>q</i>	gdmSpecListIIUserType	gdm spec list

Cadence SKILL Development Reference

Preface

Prefix	Internal Name	Data Type
<i>r</i>	defstruct	defstruct
<i>R</i>	rodObj	relative object design (ROD) object
<i>s</i>	symbol	symbol
<i>S</i>	stringSymbol	symbol or character string
<i>t</i>	string	character string (text)
<i>T</i>	txobject	transient object
<i>u</i>	function	function object, either the name of a function (symbol) or a lambda function body (list)
<i>U</i>	funobj	function object
<i>v</i>	hdbpath	—
<i>w</i>	wtype	window type
<i>x</i>	integer	integer number
<i>y</i>	binary	binary function
<i>&</i>	pointer	pointer type

For information on SKILL language, see [*Cadence SKILL Language User Guide*](#).

Additional Learning Resources

Cadence offers the following training courses on the SKILL programming language:

- [SKILL Language Programming Introduction](#)
- [SKILL Language Programming](#)
- [Advanced SKILL Language Programming](#)

For further information on these and other related Virtuoso Layout Suite training courses available in your region, visit the [Cadence Training](#) portal. You can also write to training_enroll@cadence.com.

Note: The links in this section open in a new browser. The course links initially display the requested training information for North America, but if required, you can navigate to the courses available in other regions.

How to Contact Technical Support

Cadence Customer Support is region specific. To find out the e-mail address, phone number, or fax number for your region, select the *Contacts* button from the *Cadence Online Support* home page (<http://support.cadence.com>).

You can also use *Cadence Online Support* to find out the latest information on Virtuoso Parasitic Aware Design.

Overview

- [Cadence SKILL Development Tools](#) on page 16
- [Quick Reference Tool: Finder](#) on page 17
- [Copying and Pasting Code Examples](#) on page 17

Cadence SKILL Development Tools

Information about the SKILL development tools is available in [Cadence SKILL IDE User Guide](#).

The [Walkthrough](#) topic in this help system identifies and explains the tasks you perform when you develop SKILL programs using the SKILL development tools. Using a demonstration program, it explains the various tools available to help you measure the performance of your code and also look for possible errors and inefficiencies in your code. It includes a section on working in the non-graphical environment.

For a list of [SKILL lint messages](#), and [message groups](#), refer to the *Cadence SKILL IDE User Guide*.

Quick Reference Tool: Finder

Quick reference information for syntax and abstract statements for SKILL language functions and application procedural interfaces (APIs) is available using the Finder, which is accessible from the SKILL IDE window, CIW or from the UNIX command line.



For more information, see the [*Cadence SKILL IDE User Guide*](#).

Copying and Pasting Code Examples

You can copy examples from CDSDoc windows and paste the code directly into the CIW or use the code in non-graphics SKILL mode.

To select text

- Press `Control-drag` left mouse to select a text segment of any size.

Cadence SKILL Development Reference

Overview

- Press Control-double click left mouse to select a word.
- Press Control-triple click left mouse to select an entire section.

Profiler Functions

profile

```
profile(  
    s_profileField  
)  
=> t
```

Description

Turns on global SKILL profiling for measuring time or memory.

The profiler is interrupt-driven. It walks the SKILL stack and records functions being executed. When `unprofile` or `profileSummary` are called, profiling is stopped. `profileSummary` prints a report of the time spent or memory allocated in the functions executed. Profiling time is cumulative, so you must call `profileReset` to reset the profiled data.

Time measurements are done with UNIX system functions that have very coarse granularity at 1/60 of a second, so functions must be executed many times for the CPU times to be reasonably accurate.

SKILL Profiler is not yet supported on the Windows/Wintel platform.

Cadence SKILL Development Reference

Profiler Functions

Arguments

<i>s_profileField</i>	Can be one of the following:
time	Only time is profiled.
realTime	Profiles real (elapsed) time rather than CPU time.
memory	Only SKILL memory allocated is profiled.

Value Returned

t	Always returns t.
---	-------------------

Example

```
profile( 'time)
for(i 1 10000 i+1)
profileSummary( ?file "/tmp/profile.results")
```

Reference

[unprofile](#), [profileReset](#), [profileSummary](#)

profileReset

```
profileReset(  
    )  
=> t
```

Description

Resets all SKILL profiler data.

Resets all data but keeps SKILL profiling running. Sets the accumulated CPU time and memory for all functions to zero. This is useful if you want to run the same set of profiled functions many times for different inputs so you can compare or average the results. When `profile` is first called, the profiling data is already initialized to zeros and there is no need to do an initial call to `profileReset`.

SKILL Profiler is not yet supported on the Windows/Wintel platform.

Arguments

None.

Value Returned

`t` Always returns `t`.

Example

```
profileSummary( ?file "myReport1" ) => t
```

Prints summary 1.

```
profileReset() => t
```

Resets the profiling timer/counter. Now run the same functions on another set of data.

```
profileSummary( ?file "myReport2" ) => t
```

Prints summary 2.

Reference

[profile](#)

profileSummary

```
profileSummary(  
    [ ?file t_filename ]  
    [ ?sort s_sortKey ]  
    [ ?filters g_filterSpec ]  
    [ ?maxFns x_maxDisplayed ]  
    [ ?minSecs f_minSecs ]  
    [ ?minBytes x_minBytes ]  
    [ ?children g_showChildren ]  
)  
=> t
```

Description

Prints a summary of profiling results, showing either the execution time or memory allocated to SKILL functions that were executed.

You select whether to profile time or memory by the argument you pass to the `profile` function. After executing the functions you are interested in, call `profileSummary` to generate a report of the CPU time spent in the functions or the amount of SKILL memory allocated in those functions. Using `profileSummary` options, you can sort and filter data to see only the functions in which you are interested. All functions are measured so you can create multiple profile summaries at the end of each session.

SKILL Profiler is not yet supported on the Windows/Wintel platform.

Cadence SKILL Development Reference

Profiler Functions

Arguments

<i>t_filename</i>	Specifies the report file name. Defaults to <code>ilProf.out</code> in the current working directory.				
<i>s_sortKey</i>	Changes the fields the profile summary is sorted by. <table><tr><td><code>total</code></td><td>Seconds or bytes allocated in functions and children. This is the default.</td></tr><tr><td><code>inside</code></td><td>Seconds or bytes allocated in function only.</td></tr></table>	<code>total</code>	Seconds or bytes allocated in functions and children. This is the default.	<code>inside</code>	Seconds or bytes allocated in function only.
<code>total</code>	Seconds or bytes allocated in functions and children. This is the default.				
<code>inside</code>	Seconds or bytes allocated in function only.				
<i>g_filterSpec</i>	Valid values: a regular expression, a symbol, <code>t</code> , <code>binary</code> . A regular expression displays function names indicated by the expression. For example <code>^hi</code> displays all functions beginning with <code>hi</code> . A symbol containing a context displays all functions in that context. <code>t</code> displays user functions (functions which have been loaded by the user and are not read protected). <code>'binary</code> displays only SKILL functions implemented in C.				
<i>x_maxDisplayed</i>	Integer indicating the maximum number of functions to be displayed in the profile summary.				
<i>f_minSecs</i>	Floating-point number indicating in seconds the minimum time that a function must have spent executing before it should be displayed. This time cannot be smaller than 1/60 of a second.				
<i>x_minBytes</i>	Integer that indicates the minimum number of bytes that need to be allocated to the function before it should be displayed. If both <code>?minSecs</code> and <code>?minBytes</code> are specified then any function which meets the minimum requirement of either one is displayed.				
<i>g_showChildren</i>	If <code>t</code> , then the amount of time spent in each child function and the memory allocated is printed at the bottom of the profile summary report.				

Value Returned

<code>t</code>	Always returns <code>t</code> .
----------------	---------------------------------

Cadence SKILL Development Reference

Profiler Functions

Example

```
profileSummary(?file "/tmp/summary.out"  
?sort 'inside ?children t ?maxFns 100)
```


Cadence SKILL Development Reference

Profiler Functions

unprofile

```
unprofile(  
    )  
=> t
```

Description

Turns off SKILL profiling.

Does not reset the values. `profileSummary` also turns off profiling and then prints a report.

SKILL Profiler is not yet supported on the Windows/Wintel platform.

Arguments

None.

Value Returned

t	Always returns t.
---	-------------------

Example

```
unprofile( ) => t
```

Cadence SKILL Development Reference

Profiler Functions

Lint Functions

skDisableMessage

```
skDisableMessage(  
    S_functionName  
    S_messageName  
    [ x_occurrences ]  
)  
=> t
```

Description

Disables a SKILL Lint message from being reported inside a given function definition.

Often a user does not want to disable a SKILL Lint message globally but only an individual case. You can put the call to this function in the `.skinit` file in the user's home directory or the `your_install_dir/local` directory. This function can also be inserted in the file being analyzed, outside of any function definition, before the function is defined, and SKILL Lint will recognize the call. For a list of [SKILL lint messages](#), refer to the *Cadence SKILL IDE User Guide*.

Cadence SKILL Development Reference

Lint Functions

Arguments

<i>S_functionName</i>	Function in which the given SKILL Lint error message is not reported.
<i>S_messageName</i>	Name of the message to ignore when SKILL Lint analyzes the file containing the function definition.
<i>x_occurrences</i>	Number of times to ignore the error message inside the function. This defaults to 1 if not given.

Value Returned

<i>t</i>	Always returns <i>t</i> .
----------	---------------------------

Example

```
skDisableMessage('testFunction 'Unused 1)
```

Disables the first occurrence of the SKILL Lint message `Unused` when SKILL Lint analyzes the function definition for `testFunction`.

Reference

[`sklint`](#), [`skIgnoreMessage`](#), [`skUnignoreMessage`](#)

skDisableMessageBlock

```
skDisableMessageBlock(  
    l_rules  
    g_exp1 ...  
)  
=> g_result
```

Description

Disables one or a list of SKILL Lint message(s) from being reported within the body of the this function only.

Often a user wants to turn off certain rule(s) temporarily inside a particular block of code rather than globally. You can insert the call to this function inside that particular block of code being analyzed and SKILL Lint will recognize the call.

Arguments

<i>l_rules</i>	One or a list of rule(s) to ignore for all the code within the body of this function (that is, <i>g_exp1 ...</i>).
<i>g_exp1</i>	Expressions that compose the function body.

Value Returned

<i>g_result</i>	The result of the last expression evaluated (which is essentially meaningless and can generally be ignored).
-----------------	--

Example

To disable the single SKILL Lint message for MEMBER1 from reporting (Note that the parentheses around the message name can be omitted):

```
(procedure (aFunc a b)  
    (skDisableMessageBlock MEMBER1  
        (member a b)  
        (geOpen)  
    )  
    (member 1 a)  
)
```

To disable the SKILL Lint messages for MEMBER1 and geOpen from reporting:

```
(procedure (aFunc a b)  
    (skDisableMessageBlock (MEMBER1 geOpen))  
)
```

Cadence SKILL Development Reference

Lint Functions

```
        (member a b)
      (geOpen)
    )
  (member 1 a)
)
```

Reference

[skEnableMessageBlock](#)

skEnableMessageBlock

```
skEnableMessageBlock(  
    l_rules  
    g_exp1 ...  
)  
=> g_result
```

Description

Re-enables reporting of one or a list of SKILL Lint message(s), which has/have been globally turned off within the body of the this function only.

Often a user wants to turn back on certain rule(s), which has/have been turned off globally, temporarily inside a particular block of code only rather than globally. You can insert the call to this function inside that particular block of code being analyzed and SKILL Lint will recognize the call.

Arguments

<i>l_rules</i>	One or a list of rule(s) to ignore for all the code within the body of this functionthat is, <i>g_exp1 ...</i>).
<i>g_exp1 ...</i>	Expressions that compose the function body.

Value Returned

<i>g_result</i>	The result of the last expression evaluated (which is essentially meaningless and can generally be ignored).
-----------------	--

Example

To disable the single SKILL Lint message for MEMBER1 from reporting (Note that the parentheses around the message name can be omitted):

```
(procedure (aFunc a b)  
    (skEnableMessageBlock MEMBER1  
        (member a b)  
        (geOpen)  
    )  
    (member 1 a)  
)
```

To re-enable reporting of the SKILL Lint messages for MEMBER1 and `geOpen` temporarily inside a particular block of code:

Cadence SKILL Development Reference

Lint Functions

```
(procedure (aFunc a b)
  (skEnableMessageBlock (MEMBER1 geOpen)
    (member a b)
    (geOpen)
  )
  (member 1 a)
)
```

Reference

skDisableMessageBlock

skIgnoreMessage

```
skIgnoreMessage(  
    g_ignoreList  
)  
=> t
```

Description

Turns the reporting of specified SKILL Lint messages off. For a message to appear, both the message and its group have to be unignored.

Message groups or individual messages can be ignored. For a list of [SKILL lint messages](#), and [message groups](#), refer to the *Cadence SKILL IDE User Guide*.

When a message is ignored, reporting is turned off until a call to `unignore` that same message is made. If the message group `hint` was turned off, on all subsequent runs of SKILL Lint all messages in the group `hint` would not be printed. You can put calls to `skIgnoreMessage` in `.skinit`, the SKILL Lint startup file, in either the user's home directory or under

```
your_install_dir/local
```

This startup file executes whenever you run SKILL Lint.

Arguments

<code>g_ignoreList</code>	String or list of messages that SKILL Lint will no longer output.
---------------------------	---

Value Returned

<code>t</code>	Always returns <code>t</code> .
----------------	---------------------------------

Example

```
skIgnoreMessage(' (hint))  
skIgnoreMessage("unused vars")
```

Turns off reporting of all hint and unused variable messages.

```
skUnignoreMessage(' (suggestion))
```

Turns on reporting of performance suggestion messages.

Cadence SKILL Development Reference

Lint Functions

Reference

sklint, skDisableMessage, skIgnoreMessage

Cadence SKILL Development Reference

Lint Functions

sklint

```
sklint(  
    [ ?file t_l_inputFileName ]  
    [ ?context t_contextName ]  
    [ ?outputFile t_outputFileName ]  
    [ ?ignoreGroups l_ignoreGroups ]  
    [ ?globals l_globals ]  
    [ ?depends l_depends ]  
    [ ?rulesFile t_rulesFile ]  
    [ ?ignores l_ignoresMessageList ]  
    [ ?checkNlambda g_checkNlambda ]  
    [ ?noPrintLog g_noPrintLog ]  
    [ ?useGlobalIgnores g_useGlobalIgnores ]  
    [ ?useGlobalRulesFileList g_useGlobalRulesFileList ]  
    [ ?useDisableMessages g_useDisableMessages ]  
    [ ?checkCdsFuncs g_checkCdsFuncs ]  
    [ ?checkPvtFuncs g_checkPvtFuncs ]  
    [ ?checkPubFuncs g_checkPubFuncs ]  
    [ ?prefixes l_prefixList ]  
    [ ?checkCdsPrefixes g_checkCdsPrefixes ]  
    [ ?checkFuncPrefixes g_checkFuncPrefixes ]  
    [ ?tabulate g_tabulate ]  
    [ ?skPath t_skPath ]  
    [ ?codeVersion t_release ]  
)  
=> t / nil
```

Description

Checks a SKILL file or context and reports potential errors and ways to clean up your code.

SKILL Lint checks a SKILL file or context and reports potential errors and ways to clean up your code. In particular SKILL Lint is very useful for helping programmers find unused local variables, global variables that should be locals, functions that have been passed the wrong number of arguments, and hints about how to improve the efficiency of the user's SKILL code.

SKILL Lint is generally run over a file. If a context is specified and the file is `startup.il` or is not specified, all the files ending with `.il` or `.ile` in the directory `your_install_dir/pvt/etc/context/t_contextName` are checked. By default, the SKILL Lint output prints to the Command Interpreter Window but can be printed to an output log file as well or instead. SKILL Lint prints messages about the user's code starting with the file and function name to which the message pertains. For a list of [SKILL lint messages](#), refer to *Cadence SKILL IDE User Guide*.

Cadence SKILL Development Reference

Lint Functions

Arguments

<i>tl_fileFileName</i>	The name of the file to be processed, or a list of file names. Each file is read and processed in turn. This option defaults to <code>startup.il</code> .
<i>t_context</i>	The name of the context, or an absolute-pathed context name, being processed. SKILL Lint looks under the <code>install_dir/pvt/etc/context/t_contextName</code> directory for all files ending with <code>.il</code> and <code>.ile</code> unless a file other than <code>startup.il</code> is given. If a file other than <code>startup.il</code> is given along with a context, that file is assumed to belong in that context and global variable package prefixes for that context are used if possible.
<i>t_outputFile</i>	The name of the reporting log file. Defaults to <code>context-Name.log</code> .
<i>l_ignoreGroups</i>	The list of rule groups that should not be carried out.
<i>l_globals</i>	The list of allowed globals not covered by the standard global list and the prefix list. This allows handling of obscure globals cases.
<i>l_depends</i>	The list of contexts on which the code under analysis depends. This is used for loading external definitions files.
<i>t_rulesFile</i>	The name of an additional rules file to be read prior to processing the code. For information on how to write a skLint rules file, see the <i>Cadence SKILL IDE User Guide</i> .
<i>l_ignoresMessageList</i>	The list of message IDs to ignore. These messages are not printed by SKILL Lint but are still counted in the summary report at the end of the run. Message groups as well as individual messages can be ignored. For example, all messages about improving the efficiency of SKILL code can be turned off by passing in the list <code>'(hint suggestion)</code> . For a list of <u>SKILL lint messages</u> , and <u>message groups</u> , refer to the <i>Cadence SKILL IDE User Guide</i> .
<i>g_checkNlambda</i>	Specifies whether to check the arguments to <code>nlambda</code> functions. This option should only be used by very experienced users, as it generally leads to results that are difficult to interpret. This option defaults to <code>nil</code> .
<i>g_noPrintLog</i>	Controls whether printing to the screen/ <code>ciw</code> should take place. Even if switched off, printing of start and stop messages will take place. This option defaults to <code>nil</code> .

Cadence SKILL Development Reference

Lint Functions

<i>g_useGlobalIgnores</i>	Controls whether to ignore those message IDs listed in the global variable <i>skGlobalIgnores</i> . This option is useful when the list of messages to ignore is constant and is held in a global list somewhere. This option defaults to <i>nil</i> .
<i>g_useGlobalRulesFileList</i>	Specifies whether to use the rules file listed in the global variable <i>skGlobalRulesFiles</i> . This option defaults to <i>nil</i> .
<i>g_useDisableMessages</i>	Controls whether to turn on or off disable messages to allow integrators to override message suppression put in the code. This option defaults to <i>t</i> .
<i>g_checkCdsFuncs</i>	Specifies whether to check both Cadence private and public functions (that is, force setting both <i>checkPvtFuncs</i> and <i>checkPubFuncs</i> to <i>t</i>). This option defaults to <i>nil</i> .
<i>g_checkPvtFuncs</i>	Controls whether to check Cadence private functions. This option defaults to <i>nil</i> .
<i>g_checkPubFuncs</i>	Specifies whether to check Cadence public functions. This option defaults to <i>nil</i> .
<i>l_prefixes</i>	The list of symbols whose print names are matched with variable names. The list may consist of functions and global variables not covered by the standard global list and the prefix list. This allows for obscure cases of globals to be handled.
<i>g_checkCdsPrefixes</i>	Specifies whether the prefix checking is for Cadence public function/variables start with a lower-case character. If this argument is not set to <i>t</i> (that is, by default), the checking is for customers' function/variables prefixes start with an upper-case character. Note that this option is for Cadence internal use only. This option defaults to <i>nil</i> .
<i>g_checkFuncPrefixes</i>	Controls whether function prefixes should also be checked. If this argument is not set to <i>t</i> (that is, by default), only customers' global variable prefixes are checked. This option defaults to <i>nil</i> .
<i>g_tabulate</i>	Controls whether to tabulate all the functions being called. This option defaults to <i>nil</i> .
<i>t_skPath</i>	The user-specified SKILL path to the file to be processed. If the option is specified, SKILL Lint will only search this path. Otherwise, the home directory will be searched first by default.

Cadence SKILL Development Reference

Lint Functions

t_release

The release version of code being checked (for example, 500 for IC5.0.0). If this argument is specified all automatically generated function change messages (from `cdsFuncs.cxt`) that are equal to or before the release specified (through this argument) will be filtered out (that is, will not be reported). By default, all automatically generated function change messages (from `cdsFuncs.cxt`) will be reported.

This argument is useful when the user wants to restrict reporting of function change messages which occurred after the release for which the code being checked was written. When users check the code in IC500 they will not be interested in seeing the information about the change in IC445, since that was before they wrote the code (or perhaps before it was migrated).

Specifying this argument will only filter out function changed messages. Function deleted messages will always be reported.

Value Returned

<code>t</code>	If SKILL Lint passed 100%.
<code>nil</code>	If SKILL Lint failed. SKILL Lint fails if there are any error or warning messages.

Example

```
sklint(?file "~/testfns.il")
```

Runs SKILL Lint over the `testfns.il` file and prints the output to the CIW.

```
sklint(?context "dbRead")
```

Runs SKILL Lint over all files loaded by `your_install_dir/pvt/etc/context/dbRead/startup.il`.

```
sklint(?file "~/testfns.il" ?outputFile "~/testfns.lint" ?noPrintLog t)
```

Runs SKILL Lint over the `testfns.il` file and prints the output to the `testfns.lint` file but not to the CIW.

```
sklint(?file "~/testfns.il" ?prefixes '(tfns) ?ignores '(hint suggestion))
```

Cadence SKILL Development Reference

Lint Functions

Runs SKILL Lint over the `testfns.il` file and treats all global variables that start with the prefix `tfns` as acceptable global variables. In the above example, SKILL Lint does not print any hints or suggestions for how to make the user's SKILL code more efficient.

Runs SKILL Lint to check prefixes:

```
sklint( ?file "file.il" ?prefixes '(Pre MIX) )
```

Checks non-Cadence variable prefixes only.

```
sklint( ?file "file.il" ?checkFuncPrefixes t ?prefixes '(Pre MIX) )
```

Checks both non-Cadence function and variable prefixes.

```
sklint( ?file "file.il" ?checkCdsPrefixes t ?prefixes '(le ge) )
```

Checks Cadence variable prefixes only.

```
sklint( ?file "file.il" ?checkCdsPrefixes t ?checkFuncPrefixes t ?prefixes '(le ge) )
```

Checks both Cadence function and variable prefixes.

Runs SKILL Lint on code that contain macros:

```
sklint ?file "dep.il file.il" ?depends '("dep.il")
```

where `dep.il` should contain macro definition(s) only, while `file.il` contains the code that call the macro(s) as defined in `dep.il`. Note that the order of the files specifying in the `?file` option is important that the file(s) contain the macro definition(s) have to be specified first. (that is, `dep.il` must be listed before `file.il`).

Note: Use this command for linting code where the macro definition(s) are contained in different file(s).

```
sklint ?file "file2.il" ?depends '("file2.il")
```

where `file2.il` contains both the macro definition(s) and the code that call the macro(s). Note that the macro definition(s) code have to be placed on top of the code that call the macro(s) inside `file2.il`.

Note: Use this command for linting code where the macro definition(s) and the code that call the macro(s) are contained in the same file.

Reference

[skDisableMessage](#), [skIgnoreMessage](#)

skUnignoreMessage

```
skUnignoreMessage(  
    g_ignoreList  
)  
=> t
```

Description

Turns the reporting of specified SKILL Lint messages on. For a message to appear, both the message and its group have to be unignored.

Message groups or individual messages can be unignored. For a list of [SKILL lint messages](#), and [message groups](#), refer to the *Cadence SKILL IDE User Guide*.

When a message is ignored, reporting is turned off until a call to unignore that same message is made. If the message group `hint` was turned off, on all subsequent runs of SKILL Lint all messages in the group `hint` would not be printed. You can put calls to `skIgnoreMessage` in `.skinit`, the SKILL Lint startup file, in either the user's home directory or under the `your_install_dir/local` directory. This startup file executes whenever you run SKILL Lint.

Arguments

<code>g_ignoreList</code>	String or list of messages that SKILL Lint will again output.
---------------------------	---

Value Returned

<code>t</code>	Always returns <code>t</code> .
----------------	---------------------------------

Example

```
skIgnoreMessage(' (hint))  
skIgnoreMessage("unused vars")
```

Turns off reporting of all hint and unused variable messages.

```
skUnignoreMessage(' (suggestion))
```

Turns on reporting of performance suggestion messages.

Reference

[skDisableMessage](#), [skIgnoreMessage](#), [sklint](#)

Cadence SKILL Development Reference

Lint Functions

Cadence SKILL Development Reference

Lint Functions

Context Functions

callInitProc

```
callInitProc(  
    t_contextName  
)  
=> t
```

Description

Calls all the initialization functions associated with a context.

Takes the same argument as `loadContext` (but without the `.cxt` extension) and causes all the initialization functions associated with the given context to be called. This function need not be used if the loading of the context is happening through the autoload mechanism. Use this function only when calling `loadContext` manually.

Arguments

<i>t_contextName</i>	Name of the context.
----------------------	----------------------

Value Returned

<i>t</i>	Returns <i>t</i> when initialization functions have been successfully called.
----------	---

Example

```
loadContext("myContext.cxt") => t  
callInitProc("myContext")   => t
```

All functions defined through `defInitProc` and `defUserInitProc` are called.

Cadence SKILL Development Reference

Context Functions

Reference

loadContext, defInitProc, defUserInitProc

callUserAutoInitProc

```
callUserAutoInitProc(  
    t_contextName  
)  
=> t | init_function_result
```

Description

Calls the `autoinit` function for the given context name.

If there is no `autoinit` function for the given context name, the function does not do anything and returns `t`.

Arguments

<code>t_contextName</code>	Name of the context.
----------------------------	----------------------

Value Returned

<code>g_result</code>	Returns the result of the <code>autoinit</code> function.
<code>t</code>	<code>t</code> otherwise.

Example

```
callUserAutoInitProc("fake")    => t
```

checkContextBit

```
checkContextBit(  
    t_contextPath  
)  
=> t_type
```

Description

Checks and returns the context type of the specified context file.

Arguments

<i>t_contextPath</i>	Full path to the SKILL context file.
----------------------	--------------------------------------

Value Returned

<i>t_type</i>	Returns a string (either "32bit" or "64bit") representing the context type. If the specified file is not a context file, the function displays an error message.
---------------	--

Example

```
checkContextBit("./myContext.cxt")  
=> "32bit"
```

defCapDepends

```
defCapDepends (  
    s_context  
    l_dependsList  
)  
=> t
```

Description

Specifies which contexts depend on which other contexts.

This can be specified in the `.skinit` file, which must reside in either the user's home directory or `your_install_dir/local` directory.

We recommend that developers put the `defCapDepends` function call in the beginning of the `startup.il` file because if SKILL Lint sees this call while analyzing the context it will determine the dependent contexts. When SKILL Lint is run on the context `s_context`, it loads all the dependent contexts from which it will be able to effectively type check the function calls made by the context being analyzed.

Arguments

<i>s_context</i>	Context that depends on the contexts specified by <i>l_dependsList</i> .
<i>l_dependsList</i>	List of contexts upon which <i>s_context</i> depends. In other words, <i>l_dependsList</i> should contain all the definitions for all the functions called in <i>s_context</i> that are not defined in <i>s_context</i> .

Value Returned

<i>t</i>	Always returns <i>t</i> .
----------	---------------------------

Example

```
defCapDepends ('myContext ' (skillCore hiBase))
```

Tells SKILL Lint that `myContext` depends on the contexts `skillCore` and `hiBase`.

defCapPrefixes

```
defCapPrefixes (
    s_context
    l_prefixList
)
=> t
```

Description

Specifies which prefixes are acceptable for a context's global variables.

All global variables that do not start with the stated prefixes are reported as unrecognized global variables. The call to `defCapPrefixes` can be specified in the `.skinit` file, which must reside in either the user's home directory or the `your_install_dir/local` directory.

We recommend that developers put the `defCapPrefixes` function call in the beginning of the `startup.il` file because if SKILL Lint sees this call while analyzing the context, it can determine the acceptable prefixes for the context being analyzed. If you want SKILL Lint to recognize the package prefixes when analyzing a file that contains `defCapPrefixes`, be sure to pass the `s_context` to SKILL Lint as the `?context`.

Arguments

<code>s_context</code>	Context for which the package prefixes should apply.
<code>l_prefixList</code>	List of acceptable package prefixes for <code>s_context</code> . The first letter following the prefix must be a capital. If the prefixes have been specified for a context, many of the unrecognized global variables will probably be variables that the user forgot to declare as locals.

Value Returned

<code>t</code>	Always returns <code>t</code> .
----------------	---------------------------------

Example

```
defCapPrefixes('myContext '(my))
```


Cadence SKILL Development Reference

Context Functions

Tells SKILL Lint to expect any global variables in `myContext` to start with `my` or `_my` and to report all other global variables. Thus `myGlobalVariable` is an example of a legal global variable inside `myContext`.

defInitProc

```
defInitProc(  
    t_contextName  
    s_procName  
)  
=> t
```

Description

Registers a function that the system calls immediately after autoloading a context.

When a context is autoloaded, it is given a chance to perform initialization before control returns to top level. It is during such an initialization that session-dependent objects like ports can be regenerated. This function permits a predefined function *s_procName* to be called whenever the context *t_contextName* is loaded.

Arguments

<i>t_contextName</i>	Name of context.
<i>s_procName</i>	Predefined function to be called when <i>t_contextName</i> is loaded.

Value Returned

<i>t</i>	Always returns <i>t</i> when set up. The function is not actually called at this point, but is called when the context <i>t_contextName</i> is autoloaded.
----------	--

Example

```
defInitProc("myContext" 'myInit)=> t
```

Reference

[defCapPrefixes](#), [defCapDepends](#), [setContext](#), [saveContext](#)

isContextLoaded

```
isContextLoaded(  
    t_cxt  
)  
=> t / nil
```

Description

Returns `t` if a context file with the given base name has been loaded into the current session.

Returns `nil` otherwise.

Arguments

<code>t_cxt</code>	Base name of the context file you want load status on.
--------------------	--

Value Returned

<code>t</code>	The given context has already been loaded into the current environment.
<code>nil</code>	The given context has not been loaded yet. Registers a new top-level error-handler. This error-handler is called after <code>stacktrace</code> , when an error occurs. If an error-handler already exists, the function displays a warning message. Registers a new top-level error-handler. This error-handler is called after <code>stacktrace</code> , when an error occurs. If an error-handler already exists, the function displays a warning message.

Example

```
isContextLoaded( "skillCore" ) => tRegisters a new top-level error-handler. This  
error-handler is called after stacktrace, when an error occurs. If an error-handler  
already exists, the function displays a warning message.
```

```
isContextLoaded( "hiBase" ) => nil
```

Reference

[loadContext](#)

Cadence SKILL Development Reference

Context Functions

Registers a new top-level error-handler. This error-handler is called after `stacktrace`, when an error occurs. If an error-handler already exists, the function displays a warning message.

loadContext

```
loadContext(  
    t_contextFileName  
)  
=> t / nil / error
```

Description

Loads a context file into the current session.

This function uses the SKILL path to find *t_contextFileName* if you do not supply the full path.

Prerequisites

t_contextFileName must have been created using the function `saveContext`.

Arguments

t_contextFileName Name of the context file you want to load.

Value Returned

<i>t</i>	The context was successfully loaded.
<i>nil</i>	Context has already been loaded.

Signals an error if

- The system failed to open a file
- Virtual memory is exhausted
- The version of context is incompatible with current software.

This condition usually requires that you regenerate the context file

Example

```
loadContext( "geView.cxt" )
```

Loads `geView.cxt` assuming it is in a directory in the SKILL path.

Cadence SKILL Development Reference

Context Functions

Reference

getSkillPath, load, saveContext, setContext

loadTopContextForms

```
loadTopContextForms (
    t_FileName
    [ ?debugMode g_debugMode ]
    [ ?writeProtect g_writeProtect ]
    [ ?writeProtectAll g_writeProtectAll ]
    [ ?lazyComp g_lazyComp ]
    [ ?printinfix g_printinfix ]
    [ ?integermode g_integermode ]
    [ ?mergemode g_mergemode ]
    [ ?readProtect g_readProtect ]
)
=> t / nil
```

Description

Loads top-level SKILL or Scheme forms from a file. If the `setContext` mode is set, these forms are saved in a context. After the context is loaded, these forms are evaluated at the top-level, as if these were loaded from an `.il` or `.ils` file.

Cadence SKILL Development Reference

Context Functions

Arguments

<i>t_FileName</i>	Name of the context file you want to load.
<i>g_debugMode</i>	Set debug mode
<i>g_writeProtect</i>	Set write protect
<i>g_wirteProtectAll</i>	Set write protect all
<i>g_lazyComp</i>	Set the lazy compilation status
<i>g_printinfix</i>	Set the printinfix status
<i>g_integermode</i>	Set the integer mode
<i>g_mergeMode</i>	Set the merge mode
<i>g_readProtect</i>	set the read protect mode

Value Returned

<i>t</i>	If the file is loaded successfully.
<i>nil</i>	If the file is not loaded.

Example

```
setContext("testc.cxt")
loadTopContextForms("./testFile.il")
saveContext("testc.cxt")
```


saveContext

```
saveContext (
    t_contextFileName
)
=> t / nil
```

Description

Saves the current state of the SKILL language interpreter as a binary file. This function must be used in conjunction with `setContext`.

Saves all function and variable definitions that occur, usually due to file loading, between the calls to `setContext` and `saveContext`. Those definitions can then be loaded into a future session much faster in the form of a context using the `loadContext` function.

By default all functions defined in a context are read and write protected unless the `writeProtect` system switch was turned off (by setting (`sstatus writeProtect nil`)) when the function in question was defined between the calls to `setContext` and `saveContext`.

Arguments

<code>t_contextFileName</code>	Name of binary file to which the current state of the interpreter is written.
--------------------------------	---

Value Returned

<code>nil</code>	If the saving process failed due to one of the following conditions: failed to open/create a file, exhaustion of virtual memory, presence of bad objects (such as port, db handles, and so forth)
<code>t</code>	Context was successfully saved.

Example

```
setContext ( "current")      => t
load("mySkillCode.il")      => t
defInitProc("current" 'myInit) => t
saveContext("myContext.cxt") => t
```

Cadence SKILL Development Reference

Context Functions

Reference

`defInitProc`, [`loadContext`](#), [`setContext`](#)

setContext

```
setContext (
    t_contextName
)
=> t
```

Description

Allows contexts to be saved incrementally, creating micro contexts from a session's SKILL context.

To understand this, think of the SKILL interpreter space as being linear; the function call `setContext` sets markers along the linear path. Any SKILL files loaded between a `setContext` and a `saveContext` are saved in the file named in the `saveContext` call. This function can be used more than once during a session.

Arguments

<code>t_contextName</code>	Name used to identify a context.
----------------------------	----------------------------------

Value Returned

<code>t</code>	Always returns <code>t</code> .
----------------	---------------------------------

Example

```
setContext( "current")      => t
load("mySkillCode.il")     => t
defInitProc("current" 'myInit) => t
saveContext("myContext.cxt") => t
```

Reference

`defInitProc`, [`loadContext`](#), [`saveContext`](#)

setSaveContextVersion

```
setSaveContextVersion(  
    x_newVers  
)  
=> x_oldVers
```

Description

Resets the current `saveContext` version to *x_newVers* and returns the previous context version. If *x_newVers* has an unsupported value or the function is called between `setContext` and `saveContext`, it returns an error.

Arguments

<i>x_newVers</i>	Specifies the new context version.
------------------	------------------------------------

Value Returned

<i>x_oldVers</i>	Returns the old context version.
------------------	----------------------------------

Example

```
setSaveContextVersion(getCompatContextVersion())  
601  
setSaveContextVersion(0)  
*Error* setSaveContextVersion: unsupported context version - 0
```

getCurSaveContextVersion

```
getCurSaveContextVersion(  
    )  
    => x_curVers
```

Description

Returns the current `saveContext` version (the version which the new context will have.) The possible return values are, 601 for compatible contexts and 602 for native contexts (for IC 6.1.6/CAT 33.00)

Arguments

None

Value Returned

x_curVers Returns the current `saveContext` version.

Example

```
setSaveContextVersion(getNativeContextVersion())  
601  
getCurSaveContextVersion()  
602
```

getNativeContextVersion

```
getNativeContextVersion(  
    )  
    => x_nativeVers
```

Description

Returns the native context version (for IC 6.1.6/CAT 33.00, the native context version is 602).

Arguments

None

Value Returned

x_nativeVers Returns the native context version.

Example

```
getNativeContextVersion()  
602
```

getCompatContextVersion

```
getCompatContextVersion(  
    )  
=> x_compatVers
```

Description

Returns the compatible context version (for IC 6.1.6/CAT 33.00, the compatible context version is 601).

Note: Do not use the 601 or 602 context version values directly in SKILL functions. Use `getCompatContextVersion`, `getNativeContextVersion`, or `getCurSaveContextVersion` instead to retrieve the values of context versions.

Arguments

None

Value Returned

<i>x_compatVers</i>	Returns the compatible context version.
---------------------	---

Example

```
getCompatContextVersion()  
601
```

Cadence SKILL Development Reference

Context Functions

Debug Functions

break

```
break(  
    )  
=> none
```

Description

Forces entry to the break handler if inserted directly into a SKILL function. The default break handler is the debugger.

This function is useful if you want to stop at a particular place in a function.

Arguments

None.

Value Returned

None.

Reference

breakpt, cont, continue, unbreakpt

breakpt

```
breakpt (
  [ u_function
  [ break_condition ] ]
)
where break_condition can be either
(break_tag
g_condition
)
or
(
(break_tag
g_condition)...
)
=> g_result
```

Description

Sets breakpoints on one or more functions or function objects.

The SKILL debugger is the default break handler and is entered automatically when a breakpoint is encountered. The functions `breakpt` and `unbreakpt` set and clear breakpoints on the given functions or function objects.

Another way to enter the break handler is to insert the `break` function directly into a SKILL function at the point desired. Once you are in the break handler, you can examine the state of the program. If the function was loaded under `debugMode` (see `installDebugger`), you can use single stepping functions such as `step`, `next`, `stepout`, or simply `continue`.

If `break_condition` is not specified, the breakpoint is called *unconditional*. The behavior of an unconditional breakpoint is as follows: if the function is read-protected or not yet defined under `debugMode`, the breakpoint is assumed to be set at the *call* point. Otherwise, it is assumed to be set at the *entry* point of the function.

Cadence SKILL Development Reference

Debug Functions

Arguments

<i>u_function...</i>	List of functions or function objects.								
<i>break_tag</i>	Valid values: <table><tr><td><i>call</i></td><td>Breaks at the calling point after evaluating all arguments in the caller's context.</td></tr><tr><td><i>entry</i></td><td>Breaks at the entry point after binding all formal parameters in the callee's context.</td></tr><tr><td><i>exit</i></td><td>Breaks at the exit point in the callee's context.</td></tr><tr><td><i>return</i></td><td>Breaks at the returning point in the caller's context.</td></tr></table>	<i>call</i>	Breaks at the calling point after evaluating all arguments in the caller's context.	<i>entry</i>	Breaks at the entry point after binding all formal parameters in the callee's context.	<i>exit</i>	Breaks at the exit point in the callee's context.	<i>return</i>	Breaks at the returning point in the caller's context.
<i>call</i>	Breaks at the calling point after evaluating all arguments in the caller's context.								
<i>entry</i>	Breaks at the entry point after binding all formal parameters in the callee's context.								
<i>exit</i>	Breaks at the exit point in the callee's context.								
<i>return</i>	Breaks at the returning point in the caller's context.								
<i>g_condition</i>	Condition expression to be evaluated in the associated break context.								

Value Returned

<i>g_result</i>	List of functions or function objects on which the breakpoints have been set.
-----------------	---

Example

```
installDebugger( )           ; Make sure debug mode is on.
breakpt( times )             ; Sets a breakpoint on times.
times(2 plus( 2 3 ))         ; Invokes the break handler.
cont( )                      ; Continue executing.
unbreakpt( times )           ; Clear the breakpoint.
```

This example sets a breakpoint, enters the break handler, continues, and finally clears the breakpoint.

```
(breakpt myFun hisFun)
(breakpt myFun1 (entry n > 5))
(breakpt myFun2 ((entry n > 5) (exit result != 0)) hisFun)
```

This example sets a conditional breakpoint.

The following example sets breakpoint on a function object.

```
(installDebugger)           ; Make sure debug mode is on
=>t
(defun test (x) (printf "test : x == %L\n" x))
=>test
(putd 'test1 (getd 'test))   ; test and test1 are the same functions
=>funobj:test
```

Cadence SKILL Development Reference

Debug Functions

```
(funcall 'breakpt (getd 'test))      ;set breakpoint on the funobject
(funcobj:test)
(test1 8)
<<< Break >>> on entering test
Entering new debug toplevel due to breakpoint:
Debug 2> cont
test : x == 8
=>t
```

Reference

break, cont, continue, installDebugger, next, step, stepout, unbreakpt

breakptMethod

```
breakptMethod(  
  [ S_name ]  
  [ l_specializer[ @before | @after | @around ] ]  
  [break_condition])  
where break_condition can be either  
  (break_tag  
   g_condition  
  )  
or  
  (  
   (break_tag  
    g_condition)...  
  )  
=> t / nil
```

Description

Sets breakpoint on the specified method's `defmethod` declaration.

If *break_condition* is not specified, the breakpoint is called *unconditional*. The behavior of an unconditional breakpoint is as follows: if the function is read-protected or not yet defined under `debugMode`, the breakpoint is assumed to be set at the *call* point. Otherwise, it is assumed to be set at the *entry* point of the function.

Cadence SKILL Development Reference

Debug Functions

Arguments

<i>S_name</i>	Specifies the name of the method on which the breakpoint needs to be set
<i>l_specializer</i>	Specifies a list of specializers of the method to set breakpoint on. It can include @before, @after, and @around qualifiers
<i>break_tag</i>	Valid values: call Breaks at the calling point after evaluating all arguments in the caller's context entry Breaks at the entry point after binding all formal parameters in the callee's context exit Breaks at the exit point in the callee's context return Breaks at the returning point in the caller's context
<i>g_condition</i>	Specifies the condition expression to be evaluated in the associated break context

Value Returned

t	List of breakpoints set on the specified method
nil	If the specified method is not defined

Example

```
breakptMethod()  
; when specified without arguments, lists all the breakpoints  
breakptMethod(S_name)  
; sets breakpoint on "S_name" function, behaves the same way as breakpt()  
breakptMethod(S_name nil)  
; sets breakpoint on the "generic" method of S_name.  
breakptMethod(S_name @before (classA classB))  
; sets breakpoint on "@before" method specialized on (classA classB) of "S_name"  
breakptMethod(S_name @after (classA t))  
; sets breakpoint on "@after" method specialized on (classA t) of "S_name"
```

clear

```
clear(  
    )  
=> t
```

Description

Clears all tracing and breakpoints.

This function undoes the effects of `tracef`, `tracep`, `tracev`, and `breakpt`.

Arguments

None.

Value Returned

`t` Always returns `t`.

Example

```
clear( ) => t
```

Untraces all functions and variables and clears all breakpoints.

Reference

`breakpt`, `tracef`, `tracep`, `tracev`

cont, continue

```
cont(  
    )  
=> no return value  
continue(  
    )  
=> no return value
```

Description

Continues execution from a breakpoint. `cont` and `continue` are identical.

Prerequisites

These functions work only within the break handler (which defaults to the SKILL debugger).

Arguments

None.

Value Returned

None.

Example

```
break( ) => puts you in the break handler  
cont( ) => exits the break handler
```

Reference

[break](#), [breakpt](#)

count

```
count(  
  [ { s_function ... | t } ]  
)  
=> g_result/t
```

Description

Counts the number of times a function has been called. This is an `nlambda` function. Returns the functions marked for counting.

Measures function call frequency and also serves as a valuable debugging aid. Both `count` and `uncount` accept a list of functions, or `t` for counting/uncounting all functions. To examine the number of times a function has been counted, call the `uncount` function. A list containing the number of times each function was called, along with the function name, is returned in the form of a list of (number functionName) pairs, such as,

```
((20 plus) (10 times) (5 greaterp))
```

The sublists are sorted by their first elements, using `sortcar`, so the most frequently executed functions always appear at the head of the list.

Arguments

`s_function ... | t` Turns on counting for the functions given, or all functions if `t`.

Value Returned

<code><i>g_result</i></code>	Returns the functions marked for counting.
<code><i>t</i></code>	If no arguments are given and all functions are being counted.

Example

```
count( plus greaterp )      ; Counts equal and greaterp  
setq( x plus( 2 3 ) )      ; Use the functions being counted.  
uncount( plus greaterp )  
=> ((1 plus) (0 greaterp))  ; Number of times each  
                           ; function was called.
```

Reference

[uncount](#)

Cadence SKILL Development Reference

Debug Functions

debugQuit

```
debugQuit(  
  )  
=> nil
```

Description

Exits one level of the SKILL debugger.

Arguments

None.

Value Returned

nil Always returns nil.

Example

```
installDebugger()            ; Installs the SKILL debugger.  
4 / 0                        ; Generates an error and enters  
                             ; the SKILL debugger.  
debugQuit()                 ; Exits the debugger.  
uninstallDebugger( )        ; No longer enters the debugger upon error.
```

Reference

installDebugger, uninstallDebugger

debugStatus

```
debugStatus(  
    )  
=> nil
```

Description

Prints the functions and variables being traced, functions that have breakpoints set, functions being counted, and the line breakpoints statistics.

Note: Line breakpoints statistics include the file name and the line number on which line breakpoints have been set.

Arguments

None.

Value Returned

nil Always returns nil.

Example

```
debugStatus() => nil
```

Returns nil and prints the debugging status of all functions. Sample output would look like:

```
Traced functions      (mytest)  
Traced variables     (nil)  
Traced properties    (nil)  
Breakpoints          (myFunction1)  
Counted functions    nil  
Line Breakpoints     (40 ("/home/deeptik/demo.il"))
```

Reference

breakpt, clear, count, tracef, tracev, tracep, unbreakpt, uncount, untrace,
untracev, untracep

dump

```
dump (
  [ x_variables ]
)
=> nil
```

Description

Prints the current value of all the local variables on the stack. Note that SKILL++ variables are not displayed by this function. For SKILL++ use `where` to see the lexical bindings on the stack.

`dump` is usually called from within the break or error handler.

Arguments

<i>x_variables</i>	Number of local variables on the stack to print, starting from the top. Defaults to printing all local variables on the stack.
--------------------	--

Value Returned

<code>nil</code>	Always returns <code>nil</code> .
------------------	-----------------------------------

Example

Suppose `/tmp/color.il` defines function `initColor`:

```
(defun initColor (object)
  (let ((colorList '(red green yellow)) color)
    (setq color (concat (get object 'color)))
    (if (memq color colorList)
        (printf "color %s initialized" (get object 'name)))
  )
)
```

Try this file in debugger:

```
installDebugger
=> t
1> load "/tmp/color.il"
=> t
1> (putprop 'object1 "green" 'color)
=> "green"
1> breakpoint (initColor (entry (null object)) concat)
=> concat (initColor)
```

Cadence SKILL Development Reference

Debug Functions

```
l> (initColor 'object1)
*** Error in routine fprintf/sprintf:
Message: *Error* fprintf/sprintf: format spec. incompatible
with data
Debug 2> dump
colorList = (red green yellow)
color = green
object = object1
nil
```

Reference

tracev, where

Cadence SKILL Development Reference

Debug Functions

gcsummary

```
gcsummary(  
    )  
=> t
```

Description

Prints a summary of memory allocation and garbage collection statistics in the current SKILL run.

Arguments

None.

Example

```
***** SUMMARY OF MEMORY ALLOCATION *****  
Maximum Process Size (i.e., voMemoryUsed) = 3589448  
Total Number of Bytes Allocated by IL = 2366720  
Total Number of Static Bytes          = 1605632
```

Type	Size	Allocated	Free	Static	GC count
list	12	339968	42744	1191936	9
fixnum	8	36864	36104	12288	0
flonum	16	4096	2800	20480	0
string	8	90112	75008	32768	0
symbol	28	0	0	303104	0
binary	16	0	0	8192	0
port	60	8192	7680	0	0
array	16	20480	8288	8192	0
wtype	16	16384	16224	0	0
TOTALS	--	516096	188848	1576960	9

User Type (ID)	Allocated	Free	GC count	
hiField	(20)	8192	7504	0
hiToggleItem	(21)	8192	7900	0
hiMenu	(22)	8192	7524	0
hiMenuItem	(23)	8192	5600	0
TOTALS	--	32768	28528	0

Bytes allocated for :

```
arrays      = 38176  
strings     = 43912  
strings(perm) = 68708  
IL stack    = 49140  
(Internal)  = 12288
```

TOTAL GC COUNT 9

----- Summary of Symbol Table Statistics -----

Total Number of Symbols = 11201

Hash Buckets Occupied = 4116 out of 4499

Cadence SKILL Development Reference

Debug Functions

Average chain length = 2.721331
t

How to Interpret the Summary Report

Column	Contains
Type	Data type names.
Size	Size of each atom representing the data type in bytes.
Allocated	Total number of bytes allocated in the pool for the data type.
Free	Number of bytes that are free and available for allocation.
Static	Memory allocated in static pools that are not subject to GC. This memory is usually generated when contexts are built. When variables are write protected, their contents are shifted to static pools.
GC Count	Number of GC cycles triggered because the pool for this data type was exhausted.

Reference

`gc`, `profile`, `profileSummary`, `needNCells`

getAllLoadedFiles

```
getAllLoadedFiles(  
    [ t_path ]  
)  
=> l_files / nil
```

Description

Returns a list of all files loaded since debug mode was turned on.

Arguments

<i>t_path</i>	Path to a SKILL file.
---------------	-----------------------

Value Returned

<i>l_files</i>	List of files.
<i>nil</i>	If no files have been loaded.

Example

```
getAllLoadedFiles()  
=> ("loop6.il" "demo.il"  
)
```

Returns a list of files loaded since debug mode was turned on.

```
getAllLoadedFiles("demo.il")  
=> ("/home/user1/demo.il" "/home/user1/loop6.il"  
)
```

Returns a list of files, with their full paths, loaded since debug mode was turned on.

Reference

[debugQuit](#), [installDebugger](#)

getCallingFunction

```
getCallingFunction(  
    [ tx_nestingLevel ]  
)  
=> s_functionName / nil
```

Description

Returns the name of the calling function or procedure at the specified level in the call stack.

Arguments

<i>tx_nestingLevel</i>	Optional argument that indicates the nesting level of the procedure of function name to be returned.
------------------------	--

Value Returned

<i>s_functionName</i>	The name of the calling function or procedure.
<i>nil</i>	If the specified level exceeds the call stack level, or the function at the specified level is unnamed, <i>nil</i> is returned.

Example

The following returns the name of the function one level up in the nest of calls. This is the same as `getCallingFunction(1)`

```
getCallingFunction()
```

The following returns the name of the current function.

```
getCallingFunction(0)
```

The following returns the name of the function one level up in the nest.

```
getCallingFunction(1)
```

The following returns the name of the function *n* levels up in the nest, or the top level function if *n-1* is the top, or *nil*.

```
getCallingFunction(n)
```

The following example returns the names of the functions in a list.

Cadence SKILL Development Reference

Debug Functions

```
(defun Child () (list (getCallingFunction 3)
                     (getCallingFunction 2)
                     (getCallingFunction 1)
                     (getCallingFunction)))
(defun Parent () (Child))
(defun GrandParent () (Parent))
(defun GreatGrandParent () (GrandParent))

(GreatGrandParent)
=> (GreatGrandParent GrandParent Parent Child)
```

getFunctions

```
getFunctions(  
    t_fileName  
)  
=> l_functions / nil
```

Description

Returns functions defined in a file loaded after debug mode is turned on.

Returns the functions that were defined the last time that the file was loaded. Only the file name should be used and not its full path. If no extension is given, `.il` is assumed.

Prerequisites

Turn on debug mode before loading the file.

Arguments

<code>t_fileName</code>	File name loaded after debug mode was turned on.
-------------------------	--

Value Returned

<code>l_functions</code>	A list of functions.
<code>nil</code>	If no functions were defined in that file or if the file was not loaded after debug mode was turned on.

Example

```
getFunctions( "testfns" )
```

Returns the functions defined in `testfns.il`.

Reference

[getAllLoadedFiles](#), [installDebugger](#)

getGFbyClass

```
getGFbyClass(  
    s_ClassName  
    [ g_nonExistent ]  
    [ g_clearGFcache ]  
)  
=> l_methods
```

Description

Returns a list of generic functions specializing on a given class.

Arguments

<i>s_ClassName</i>	Name of the class for which you want to view the list of specializing functions.
<i>g_nonExistent</i>	If set to <code>t</code> , lists all the generic functions specializing only on non-defined classes.
<i>g_clearGFcache</i>	Specifies whether to clear the method dispatch cache. The default is <code>nil</code> .

Value Returned

<i>l_methods</i>	A list of generic functions.
------------------	------------------------------

Example

```
getGFbyClass('systemObject)
```

returns

```
(printObject)
```

ilAddTopLevelErrorHandler

```
ilAddTopLevelErrorHandler(  
    [ s_handler ]  
)  
=> t
```

Description

Registers a new top-level error-handler. This error-handler is called after stacktrace, when an error occurs. If an error-handler already exists, the function displays a warning message.

Arguments

<i>s_handler</i>	A Skill function that accepts 0 or 1 argument. When an error is raised, the error message string is passed to the handler.
------------------	--

Value Returned

t	Returns t, if the error-handler is successfully registered, otherwise, throws an error.
---	---

Example

To define an error handler and then register it:

```
defun(myhandler(x)  
printf("TOP LEVEL ERROR HANDLER : %L" x) );  
ilAddTopLevelErrorHandler('myHandler)
```

ilDebugCountLevels

```
ilDebugCountLevels(  
    )  
=> x_levels
```

Description

Returns the number of top-level debug and error frames present in the SKILL stack, if SKILL is in breakpoint or error top level. Otherwise, returns 1.

Arguments

None.

Value Returned

<code>x_levels</code>	Number of top-level error and debug frames present in the SKILL stack.
-----------------------	--

Example

To return the number of top-level error frames debugged in the SKILL stack:

```
ilDebugCountLevels();
```

ilGetGFbyClass

```
ilGetGFbyClass(  
    s_ClassName  
    [ g_nonExistent ]  
    [ g_clearGFcache ]  
)  
=> l_methods
```

Description

Returns a list of generic functions specializing on a given class.

Note: An alias to this function with the name getGFbyClass exists.

Arguments

<i>s_ClassName</i>	Name of the class for which you want to view the list of specializing functions.
<i>g_nonExistent</i>	If set to <code>t</code> , lists all the generic functions specializing only on non-defined classes.
<i>g_clearGFcache</i>	Specifies whether to clear the method dispatch cache. The default is <code>nil</code> .

Value Returned

<i>l_methods</i>	A list of generic functions.
------------------	------------------------------

Example

```
ilGetGFbyClass('systemObject)
```

returns

```
(printObject)
```

ilGetIdeSessionWindow

```
ilGetIdeSessionWindow(  
    [ g_force ]  
)  
=> w_IDE / nil
```

Description

Returns the SKILL IDE session window (*w_IDE*), or *nil* if the IDE session window does not exist. If the optional argument *g_force* is set and is not *nil*, the function creates an IDE window and displays it on the screen.

Arguments

<i>g_force</i>	If set and is not <i>nil</i> , the function creates an IDE window and displays it on the screen.
----------------	--

Value Returned

<i>w_IDE</i>	Returns the SKILL IDE session window.
<i>nil</i>	Returns <i>nil</i> if the IDE session window does not exist.

Example

To return the SKILL IDE session window:

```
ilGetIdeSessionWindow()  
=>nil  
  
ilGetIdeSessionWindow(t)  
=>swindow:1
```


ilRemoveMethod

```
ilRemoveMethod(  
    s_genFunction  
    g_className  
    [g_method]  
)  
=> t / nil
```

Description

Removes a given method from a generic function.

Note: An alias to this function with the name removeMethod exists.

Arguments

<i>s_genFunction</i>	Name of the generic function from which the method needs to be removed.
<i>g_className</i>	Name of the class or a list of classes to which the generic function belongs.
<i>t_method</i>	Specifies the method qualifier. It can have one of the following values: '@before', '@after', and '@around'. If this value is not provided or is specified as <code>nil</code> , then the primary method is removed.

Value Returned

<code>t</code>	If the method is successfully removed.
<code>nil</code>	If the method is not removed.

Example

To remove the object `myClass` from the function `myFunction`:

```
ilRemoveMethod('myFunction 'myClass)  
ilRemoveMethod('myFunB '(classX classY) '@after)
```

ilRemoveTopLevelErrorHandler

```
ilRemoveTopLevelErrorHandler(  
    )  
=> t
```

Description

Unregisters the top-level error-handler, previously registered by the `ilAddTopLevelErrorHandler` function.

Arguments

None.

Value Returned

`t` Always returns `t`.

Example

To unregister the top-level error-handler:

```
ilRemoveTopLevelErrorHandler()
```

ilSlotBoundp

```
ilSlotBoundp(  
    obj  
    t_slotName  
)  
=> t / nil
```

Description

Checks if a named slot is bound (has been assigned a value) to an instance.

Arguments

<i>obj</i>	An instance of some class.
<i>t_slotName</i>	Slot name.

Value Returned

<i>t</i>	If the slot is bound.
<i>nil</i>	If the slot is unbound.

Note: It throws an error if *obj* or *t_slotName* is invalid.

Example

```
myObject->slotX = 20  
ilSlotBoundp(myObject "slotX")=>t
```

ilToolBox

```
ilToolBox(  
    )  
=> t / nil
```

Description

Brings up the SKILL Development toolbox.

SKILL Development is a toolbox for debugging SKILL programs using a form-based graphical user interface.

Arguments

None.

Value Returned

t Always returns *t*.

Example

```
ilToolBox () =>t
```

inNext

```
inNext (
    )
    => t / nil
```

Description

Returns `t` if the function is called in the expression that is executed in the debugged code on the `next()` SKILL function.

Arguments

None.

Value Returned

<code>t</code>	If the current debugged expression is executed on <code>next()</code> .
<code>nil</code>	If the current debugged expression is not executed on <code>next()</code> .

Example

```
installDebugger()
Loading skillDev.cxt
t
1> defun( callInNext () printf("inNext() => %N\n" inNext())
)
callInNext
1> defun( test (x)
    x = list(x x)
    callInNext()
    x = list(x x)
)
test
1> breakpoint test
(test)
1> test(4)
<<< Break >>> on entering test
Entering new debug toplevel due to breakpoint:
Type (help "debug") for a list of commands or debugQuit to exit the toplevel.
```

Cadence SKILL Development Reference

Debug Functions

```
Debug 2> next
stopped before evaluating (x = list(x x))
Debug 2> next
stopped before evaluating callInNext()
Debug 2> next
inNext() => t
stopped before evaluating (x = list(x x))
Debug 2> callInNext()
inNext() => nil
t
Debug 2> cont
((4 4)
  (4 4)
)
```

inStepOut

```
inStepOut(  
    )  
=> t / nil
```

Description

Returns `t` if the function is called in the `stepout()` SKILL function.

Arguments

None.

Value Returned

<code>t</code>	If called in <code>stepout()</code> .
<code>nil</code>	If not called in <code>stepout()</code> .

Example

```
installDebugger()  
Loading skillDev.cxt  
t  
1> breakpt test  
(test)  
1> defun( callInStepOut () printf("inStepOut() => %N\n" inStepOut()))  
callInStepOut  
1> defun( test (x)  
x = list(x x)  
callInStepOut()  
)  
function test redefined  
test  
1> test(4)  
<<< Break >>> on entering test  
Entering new debug toplevel due to breakpoint:  
Debug 2> callInStepOut()  
inStepOut() => nil  
t  
Debug 2> next  
stopped before evaluating (x = list(x x))  
Debug 2> callInStepOut()  
inStepOut() => nil  
t  
Debug 2> next  
stopped before evaluating callInStepOut()  
Debug 2> stepout
```

Cadence SKILL Development Reference

Debug Functions

```
inStepOut () => t  
t
```


installDebugger

```
installDebugger(  
    )  
=> t / nil
```

Description

Installs the SKILL debugger as the error handler so that the debugger is entered automatically upon error. Turns on debug mode.

`installDebugger` also turns on debug mode and allows all functions, including those that are write protected, to be redefined. Debug mode stores cross-referencing information about functions and files as well as more information for stacktraces. Debug mode also changes the prompt to display the number of nested debuggers plus one. You might find it desirable to put the `installDebugger` function in your initialization file while you are developing your code.

Arguments

None.

Value Returned

<code>t</code>	If the debugger is successfully installed.
<code>nil</code>	Otherwise.

Example

```
installDebugger()      ; Installs the SKILL debugger.  
4 / 0                  ; Generates an error and enters  
                        ; the SKILL debugger.  
stacktrace             ; Displays the SKILL stack.  
debugQuit()            ; Exits the debugger.
```

Reference

[uninstallDebugger](#), [debugQuit](#), [stacktrace](#)

listAlias

```
listAlias(  
    [ s_aliasName ]  
)  
=> s_functionName / l_propertyList / nil
```

Description

Prints a (property) list of all current aliases and associated function symbols, or the function symbol that the given alias defines to.

Arguments

<i>s_aliasName</i>	Symbol name of the alias
--------------------	--------------------------

Value Returned

<i>s_functionName</i>	Function symbol that the given alias name defines to.
<i>l_propertyList</i>	Property list of all current aliases and associated function symbols (This is printed when no argument is specified).
nil	The given alias name is not an alias for any function, or there's no alias defines for any function at all.

Example

```
;; Defines 'lf' and 'e' as the aliases of the listFunctions()  
;; and edit() functions, respectively  
alias(lf listFunctions) => lf  
alias(e edit) => e  
  
;; Prints the name of the function that 'lf' aliases to  
listAlias('lf') => listFunctions  
  
;; Prints A property list of all current aliases and associated  
;; function symbols  
listAlias() => (e edit lf listFunctions)  
  
;; The given alias name is not an alias for any function  
listAlias('bogus') => nil  
  
;; Remove 'lf' and 'e' as aliases  
unalias(lf) => (lf)  
unalias(e) => (e)  
  
;; There's no alias defines for any function anymore  
listAlias() => nil
```

listFunctions

```
listFunctions(  
    t_pattern  
    [ g_listAllFuncs ]  
)  
=> l_functions / nil
```

Description

Returns all public function names that contain the given substring or match the given regular expression. If the second (optional) argument is specified as `t` (or a non-nil value), `listFunctions` would look at the SKILL Virtual Machine, rather than the `cdsFinder` database, and return all `isCallable` Cadence public functions and user-defined SKILL functions that contain the given substring or match the given regular expression.

By default, the source of the returned values is the SKILL directory in the SKILL Finder database, which is located in the doc hierarchy under `your_install_dir/doc/finder/SKILL`. `listFunctions` will also look in the `your_install_dir/local/finder/SKILL` directory, as well as directories specified in `CDS_FINDER_PATH` (refer to the “[Environment Variable for Additional Finder Data Directories](#)” section of *Cadence SKILL IDE User Guide* for usage of `CDS_FINDER_PATH`), for any personal functions you may have created and placed there in the appropriate SKILL Finder format, which is described in the toolbox help files of *Cadence SKILL IDE User Guide*.

The returned function names can be used for passing a list of function names that match a given pattern to another function.

Arguments

<code>t_pattern</code>	Pattern to search for.
<code>g_listAllFuncs</code>	If specified as <code>t</code> (or a non-nil value), all <code>isCallable</code> Cadence public and user-defined function names that contain the given substring or match the given regular expression are returned. Default is <code>nil</code> .

Value Returned

<code>l_functions</code>	All public function names that match the given <code>t_pattern</code> .
<code>nil</code>	If no functions are found that match the pattern.

Cadence SKILL Development Reference

Debug Functions

Example

```
apply( 'tracef listFunctions( "hi" ) )
```

Calls the `tracef` function with all the functions that contain the substring `hi`.

```
apply( 'tracef listFunctions( "x" t ) )
```

Calls the `tracef` function with all `isCallable` Cadence public and user-defined functions that contain the substring `"x"`

```
listFunctions("^hi")
```

Lists all the functions that begin with `"hi"`

```
listFunctions("^db.*Copy")
```

Lists all the functions that begin with `db` and have `"Copy"` in their names.

Reference

[listVariables](#), [rexCompile](#), [rexMatchp](#)

listVariables

```
listVariables(  
    t_pattern  
)  
=> l_variables
```

Description

Returns all variable names that match the given substring or regular expression as part or all of their print name.

They can be used for passing a list of variable names that match a given pattern to a function.

Arguments

<i>t_pattern</i>	Pattern to search for.
------------------	------------------------

Value Returned

<i>l_variables</i>	All variable names that match the given <i>t_pattern</i> .
--------------------	--

Example

```
apply( 'tracev listVariables( "myVars" ) )
```

Traces the variables that match the pattern `myVars`.

Reference

[listFunctions](#), [rexCompile](#)

memoryAllocated

```
memoryAllocated(  
    )  
=> f_megabytesAllocated
```

Description

Returns the amount of memory allocated by a process. The returned value is an approximation in megabytes and might not include the memory that has been allocated, but the amount that is unused.

The returned value is intended to be larger than the value returned from a previous invocation of this function, in case, a large amount of memory has been allocated.

Arguments

None.

Arguments

<i>f_megabytesAllocated</i>	Amount of memory that the process has allocated (an approximation in megabytes.)
-----------------------------	--

Example

```
memoryAllocated() => 1.743386
```

Cadence SKILL Development Reference

Debug Functions

next

```
next (
    [ x_steps ]
)
=> none
```

Description

Allows execution to proceed until the next expression. This function only works if executed from within a break handler and if the code you want to step through was loaded under debugMode. See `installDebugger`.

`next` reenters the break handler after completing *x_step* expressions, as long as the program has not finished.

You cannot execute the `next` function inside functions that are read protected.

Arguments

<i>x_steps</i>	Number of SKILL expressions to execute at or above the current stack depth.
----------------	---

Value Returned

None.

Example

Suppose `/tmp/color.il` defines function `initColor`:

```
(defun initColor (object)
  (let ((colorList '(red green yellow)) color)
    (setq color (concat (get object 'color)))
    (if (memq color colorList)
        (printf "color %s initialized" (get object 'name)))
  )
)
```

Try this file in debugger:

```
installDebugger
=> t
(ssstatus sourceTracing t)           ; Turns on sourceTracing
                                     ; to get line numbers.
=> t
```

Cadence SKILL Development Reference

Debug Functions

```
1> load "/tmp/color.il"
=> t

1> (putprop 'object1 "green" 'color)
=> "green"

1> breakpt(initColor (entry (null object)) concat)
=> concat(initColor)

1> (initColor 'object1)
<<< Break >>> on calling concat with args ("green")
at line 3 in file /tmp/color.il

Debug 2> next                                ; Proceeds to 'if' form
stopped at line 4 in file /tmp/color.il
before evaluating if(memq(color colorList) printf("color %s initialized"...

Debug 2> step                                ; Steps into 'if' form
stopped at line 4 in file /tmp/color.il
before evaluating memq(color colorList)

Debug 2> next                                ; Proceeds to 'printf' form
stopped at line 5 in file /tmp/color.il
before evaluating printf("color %s initialized" get(object 'name))
```

Reference

[breakpt](#), [installDebugger](#), [step](#)

Cadence SKILL Development Reference

Debug Functions

pp

```
pp(  
    s_functionName  
    [ p_outputPort ]  
)  
=> nil
```

Description

Pretty prints the definition of a function. The function must not be read-protected. This is an `nlambda` function.

Each function definition is printed in a manner that allows it to be read back into SKILL. `pp` does not evaluate its first argument but does evaluate the second argument, if given.

Arguments

<i>s_functionName</i>	Name of the function to be pretty printed.
<i>p_outputPort</i>	Output port to print to. Default is <code>poport</code> .

Value Returned

<code>nil</code>	Pretty prints the function.
------------------	-----------------------------

Example

```
procedure(fac(n) if(n <= 1 1 n*fac(n-1)))=> fac  
pp fac  
procedure(fac(n)  
    if((n <= 1) 1  
        (n * fac(n - 1))  
    )  
)  
=> nil
```

Defines the factorial function `fac` then pretty prints it to `poport`.

Reference

`profile`, `pprint`

printFunctions

```
printFunctions(  
    t_pattern  
    [ p_outport ]  
    [ g_listAllFuncs ]  
)  
=> t
```

Description

Prints all function names that contain the given substring or match the given regular expression.

These functions are useful for finding functions that contain the same substring or finding an individual function when you know only part of the name.

By default, the source of the returned values is the SKILL directory in the SKILL Finder database, which is located in the doc hierarchy under *your_install_dir/doc/finder/SKILL*. `printFunctions` will also look in the *your_install_dir/local/finder/SKILL* directory, as well as directories specified in `CDS_FINDER_PATH` (refer to the [*Environment Variable for Additional Finder Data Directories Help*](#) section of *Cadence SKILL IDE User Guide* for usage of `CDS_FINDER_PATH`), for any personal functions you may have created and placed there in the appropriate SKILL Finder format, which is described in the toolbox help files of [*Cadence SKILL IDE User Guide*](#).

If the third (optional) argument is specified to `t`, `printFunctions` would look at the SKILL Virtual Machine, rather than the `cdsFinder` database, and returns all `isCallable` Cadence public functions and user-defined SKILL functions that contain the given substring or match the given regular expression.

Cadence SKILL Development Reference

Debug Functions

Arguments

<i>t_pattern</i>	Pattern to search for.
<i>p_outport</i>	Optional output port. Default is <code>poport</code> .
<i>g_listAllFuncs</i>	If specified to <i>t</i> , all <code>isCallable</code> Cadence public and user-defined function names that contain the given substring or match the given regular expression are returned. Default is <code>nil</code> .

Value Returned

<i>t</i>	Always returns <i>t</i> (after printing all the function names that contain the given substring or match the given regular expression).
----------	---

Example

```
printFunctions( "installDebug" ) => t
```

Returns *t* and prints all the function names that contain the substring `installDebug`.

```
printFunctions( "x" nil t)
```

Returns *t* and prints all `isCallable` Cadence public and user-defined function names that contain the substring *x*.

```
p = outfile( "outfile" ) => port:"outfile"
printFunctions( "x" p t )
```

Returns *t* and writes all `isCallable` Cadence public and user-defined function names that contain the substring *x* to outfile.

Reference

[listFunctions](#), [printVariables](#), [rexCompile](#)

printObject

```
printObject(  
    g_object  
    [ p_outputPort ]  
)  
=> g_result
```

Description

A generic function that writes a description of an object to an output port.

If you define a method for this generic function, it should call one or more of the SKILL print functions. See the *Cadence SKILL Language User Guide* for a discussion of generic functions.

Arguments

<i>g_object</i>	Object whose print representation you want.
<i>p_outputPort</i>	Specified output port.

Value Returned

<i>g_result</i>	Returns the return value of the method that was called.
-----------------	---

Example

```
P = makeInstance( 'Point ?name "P" ?x 3 ?y 4 )  
=> stdobj:0x1d003c  
ILS-<2> defmethod( printObject ((p Point)  
    @optional ( port poport ))  
    fprintf( poport "%s @ %n:%n\n" p->name p->x p->y )  
    nil  
    )  
t  
ILS-<2> printObject( P )  
P @ 3:4  
nil
```

Prints P @ 3:4, the location of P.

```
ILS-<2> defstruct( card rank suit )  
t  
ILS-<2> mycard = make_card( ?rank 2 ?suit "spades" )
```

Cadence SKILL Development Reference

Debug Functions

```
array[4]:2304000
ILS-<2> addDefstructClass( card )
funobj:0x1c98f8
ILS-<2> printObject( mycard )
Loading skillDev.cxt
Structure of type card:
  rank: 2
  suit: "spades"
t
```

Reference

`printf`, `print`, `println`, `fprintf`, `defmethod`

printstruct

```
printstruct(  
    g_object  
)  
=> t
```

Description

Prints the contents of an association table or defstruct in a tabular format.

For debugging purposes, the `printstruct` function prints the contents of a structure in an easily readable form. It recursively prints nested structures.

Arguments

g_object Defstruct or association table to be printed.

Value Returned

t Prints contents of the defstruct or association table.

Example

```
defstruct(myStruct slot1 slot2) => t  
struct = make_myStruct(?slot1 "one" ?slot2 'two)=> array[4]:3872800  
printstruct(struct)  
Structure of type myStruct:  
slot1: "one"  
slot2: two => t
```

Reference

`defstruct`, `defstructp`, `makeTable`

printVariables

```
printVariables(  
    t_pattern  
    [ p_outport ]  
)  
=> t
```

Description

Prints all variable names that contain the given substring or match the given regular expression, along with their values.

This function is useful for finding variables that contain the same substring or finding an individual variable when you know only part of the name. The `printVariables` function also prints the value of each variable it finds.

Arguments

<i>t_pattern</i>	Pattern to search for.
<i>p_outport</i>	Optional output port. The default is <code>poport</code> .

Value Returned

<i>t</i>	Always returns <i>t</i> and prints the value of each variable it finds.
----------	---

Example

```
printVariables( "stack" )  
                _stacktrace 0  
=> t
```

Prints all the variables with their values that contain the substring `stack` and returns *t*. The underscore (`_`) at the beginning of `_stacktrace` indicates that it is an internal system variable.

Reference

[listVariables](#), [printFunctions](#)

removeMethod

```
removeMethod(  
    s_genFunction  
    g_className  
    [g_role]  
)  
=> t / nil
```

Description

Removes a given method from a generic function.

Note: For compatibility with previous releases, an alias to this function with the name, `ilRemoveMethod` exists.

Arguments

<i>s_genFunction</i>	Name of the generic function from which the method needs to be removed.
<i>g_className</i>	Name of the class or a list of classes to which the generic function belongs.
<i>g_role</i>	Specifies the method qualifier. It can have one of the following values: '@before', '@after', '@around', or nil. If this value is not provided or is specified as nil, then the primary method is removed.

Value Returned

<i>t</i>	If the method is successfully removed.
<i>nil</i>	If the method is not removed.

Example

To remove the object `myClass` from the function `myFunction`:

```
removeMethod('myFunction 'myClass)  
removeMethod('myFunB '(classX classY) '@after)
```


resume

```
resume(  
    [ g_result ]  
)
```

Description

Exits the interactive top-level loop started with the most recently invoked `toplevel` function and returns its argument to the caller of `toplevel`. Do not use this function programmatically; use it only as an interactive command.

The `resume` function itself does not return. It returns value of the `toplevel` function.

- To start a top-level interactive loop in SKILL++ mode, type

```
toplevel( 'ils )
```

- To start a top-level interactive loop in SKILL mode, type

```
toplevel( 'il )
```

Arguments

<i>g_result</i>	Optional value to be returned as the result from the previous <code>toplevel</code> calls.
-----------------	--

Value Returned

Returns the return value of the `toplevel` function. The `resume` function itself does not return.

Example

Following is a transcript of a brief session, including prompts.

```
> R = topLevel( 'ils )  
ILS-<2> resume( 1 )  
1  
> R  
1
```

Reference

`toplevel`, `errset`

skillDebugger

```
skillDebugger(  
    )  
=> nil
```

Description

Activates the SKILL Debugger. Usually invoked by a break or error handler.

You do not normally call it; instead it is invoked by the break or error handler. The SKILL debugger is the default break handler, and can also be used as the current error handler by calling the `installDebugger` function.

When you enter the debugger, the prompt changes to `debug #>` where # is a number identifying the number of nested debuggers plus one. Once in the SKILL debugger, you can examine the stack and local variables with functions such as `stacktrace`, `dump`, and `where`. You can also execute any SKILL function normally available because the debugger calls the SKILL top level. To quit the function, use `debugQuit`.

If the SKILL debugger is entered from a breakpoint, the following functions can be used to resume execution: `step`, `next`, `stepout`, and `continue`. If an error occurs and an `errset` is on the stack, the SKILL Debugger will not be invoked. To debug errors in this case, set `_stacktrace` to an integer value greater than zero or set breakpoints before the error occurs.

Arguments

None.

Value Returned

`nil` Always returns `nil`.

Example

```
skillDebugger()      ; Calls the debugger.  
debugQuit()          ; Exits the debugger.  
alias q debugQuit    ; Alias used for faster typing.
```

Cadence SKILL Development Reference

Debug Functions

Reference

debugQuit, dump, cont, continue, installDebugger, next, stacktrace, step, stepend,
stepout, where, uninstallDebugger

skillDevStatus

```
skillDevStatus(  
    )  
=> t/nil
```

Description

Returns the current status of the SkillDev license.

Arguments

None.

Value Returned

t	SkillDev license is checked out.
nil	SkillDev license is not checked out.

Example

```
skillDevStatus()  
=> nil
```

stacktrace

```
stacktrace(  
    [ g_unevaluated ]  
    [ x_depth ]  
    [ x_skip ]  
    [ p_port ]  
)  
=> x_result
```

Description

Prints the functions on the stack and their arguments to the depth specified, or to the bottom of the stack.

Observes the following rules:

- When debug mode is on `stacktrace`, it prints the evaluated function arguments by default if the status switch `traceArgs` has been set to `t`.
- When debug mode is off, `stacktrace` always prints the unevaluated arguments.
- If the status switch `stacktrace` is set (using the `sstatus` function) to an integer, it prints that number of stack frames automatically whenever an error occurs.
- If there are no functions on the stack, that is, you are at the very top-level, then `stacktrace` does not print anything and returns 0.

Note: `stacktrace` has a more flexible interface for user convenience. Thus if the first argument is a number it will interpret it to be `x_depth`, otherwise if it is non-`nil` it will take it to be `g_unevaluated`.

Prerequisites

This function is generally used inside the break or error handler.

Cadence SKILL Development Reference

Debug Functions

Arguments

<i>g_unevaluated</i>	If <i>t</i> , always prints the unevaluated function parameters.
<i>x_depth</i>	Number of stack levels to print, defaults to all.
<i>x_skip</i>	<code>stacktrace</code> skips over the number of function calls specified by <i>x_skip</i> . This argument defaults to 1.
<i>p_port</i>	Port for the <code>stacktrace</code> output, defaults to the error port.

Value Returned

<i>x_result</i>	The number of stack frames printed.
-----------------	-------------------------------------

Example

```
stacktrace()
```

Prints all the functions on the stack.

```
stacktrace( 5 )
```

Prints the top five functions on the stack.

```
stacktrace( t 5 3 ptport )
```

Prints the five functions on the stack that come after the first three to the trace port.

```
sstatus( stacktrace 6 )
```

Prints the first six stack frames every time an error occurs.

Reference

[breakpt](#), [cont](#), [continue](#), [dump](#), [installDebugger](#), [next](#), [sstatus](#), [step](#), [stepend](#), [stepout](#), [unbreakpt](#), [uninstallDebugger](#), [where](#)

step

```
step(  
    [ x_steps ]  
)
```

Description

Steps into functions and executes a given number of SKILL functions. This function only works if executed from within a break handler and if the code you want to step through was loaded under `debugMode`. See [installDebugger](#).

The number of steps defaults to 1 if there is no argument given. After completing `x_steps`, `step` re-enters the break handler before executing its next function, as long as the program has not finished. You cannot step inside functions that are read protected.

Arguments

<code>x_steps</code>	Number of SKILL commands to execute.
----------------------	--------------------------------------

Value Returned

None.

Example

Suppose `/tmp/color.il` defines function `initColor`:

```
(defun initColor (object)  
  (let ((colorList '(red green yellow)) color)  
    (setq color (concat (get object 'color)))  
    (if (memq color colorList)  
        (printf "color %s initialized" (get object 'name)))  
  )  
)
```

Try this file in debugger:

```
installDebugger  
=> t  
  
1> (sstatus sourceTracing t)  
                                ; Turns on sourceTracing to get line numbers  
=> t
```

Cadence SKILL Development Reference

Debug Functions

```
1> load "/tmp/color.il"
=> t
```

```
1> (putprop 'object1 "green" 'color)
=> "green"
```

```
1> breakpt(initColor (entry (null object)) concat)
=> concat(initColor)
```

```
1> (initColor 'object1)
<<< Break >>> on calling concat with args ("green")
at line 3 in file /tmp/color.il
```

```
Debug 2> next
stopped at line 4 in file /tmp/color.il
before evaluating if(memq(color colorList) printf("color %s initialized"...
```

```
Debug 2> step
stopped at line 4 in file /tmp/color.il
before evaluating memq(color colorList)
```

```
Debug 2> next
stopped at line 5 in file /tmp/color.il
before evaluating printf("color %s initialized" get(object 'name))
```

```
Debug 2> step
stopped at line 5 in file /tmp/color.il
before evaluating get(object 'name)
```

Reference

breakpt, cont, continue, dump, next, stepend, stepout, unbreakpt, where

stepend

```
stepend(  
    [ x_stepN ]  
)
```

Description

Allows execution to proceed to the end of the `n`th enclosing form and displays its result. `stepend` cannot proceed past the end of the current function. This function only works if executed from within a break handler and if the code you want to step through was loaded under `debugMode`. See [installDebugger](#) on page 97.

Comparing `step` and `stepend`:

- `step` proceeds to the “beginning” of the $(n + 1)$ th possibly enclosed form and displays the next form to be evaluated.
- `stepend` proceeds to the end of the `n`th enclosing form and displays its result. `stepend` cannot proceed past the end of the current function.

Arguments

<code>x_stepN</code>	Number of forms to step through.
----------------------	----------------------------------

Value Returned

None.

Example

Suppose `/tmp/color.il` defines function `initColor`:

```
(defun initColor (object)  
  (let ((colorList '(red green yellow)) color)  
    (setq color (concat (get object 'color)))  
    (if (memq color colorList)  
        (printf "color %s initialized" (get object 'name)))  
  )  
)
```

Try this file in debugger:

```
installDebugger  
=> t
```

Cadence SKILL Development Reference

Debug Functions

```
1> (sstatus sourceTracing t)
; Turns on sourceTracing to get line numbers
=> t

1> load "/tmp/color.il"
=> t

1> (putprop 'object1 "green" 'color)
=> "green"

1> breakpt(initColor (entry (null object)) concat)
=> concat(initColor)

1> (initColor 'object1)
<<< Break >>> on calling concat with args ("green")
at line 3 in file /tmp/color.il

Debug 2> next
stopped at line 4 in file /tmp/color.il
before evaluating if(memq(color colorList) printf("color %s initialized"...

Debug 2> step
stopped at line 4 in file /tmp/color.il
before evaluating memq(color colorList)

Debug 2> stepend
stopped at line 4 in file /tmp/color.il
after evaluating memq(color colorList)
==> (green yellow)

Debug 2> stepend
stopped at line 5 in file /tmp/color.il
after evaluating get(object 'name)
==> nil
```

Reference

step, stepout

stepout

```
stepout(  
    [ x_steps ]  
)
```

Description

Allows execution to proceed until the evaluator returns from the current function.

It reenters the break handler when the current function returns to its caller.

Arguments

<i>x_steps</i>	Number of function call levels to return from before reentering the break handler. Defaults to 1.
----------------	---

Value Returned

None.

Example

Suppose `/tmp/color.il` defines function `initColor`:

```
(defun initColor (object)  
  (let ((colorList '(red green yellow)) color)  
    (setq color (concat (get object 'color)))  
    (if (memq color colorList)  
        (printf "color %s initialized" (get object 'name)))  
    )  
)
```

Try this file in debugger:

```
installDebugger  
=> t  
  
1> (sstatus sourceTracing t)  
                                ; Turns on sourceTracing to get line numbers  
=> t  
  
1> load "/tmp/color.il"  
=> t  
  
1> (putprop 'object1 "green" 'color)  
=> "green"
```

Cadence SKILL Development Reference

Debug Functions

```
=> initColor

1> (putprop 'object1 "green" 'color)
=> "green"

1> breakpt(initColor (entry (null object)) get)
=> (get initColor)

1> (initColor 'object1)
<<< Break >>> on calling get with args (object1 color)
at line 3 in file /tmp/color.il

Debug 2> stepout
<<< Break >>> on calling get with args (object1 name)
at line 5 in file /tmp/color.il
                                ; stop at next 'get'
```

Reference

breakpt, cont, continue

toplevel

```
toplevel(  
  [ s_langMode ]  
  [ e_envobj ]  
)  
=> g_result
```

Description

Starts an interactive top-level loop in either SKILL or SKILL++ mode.

All expressions you enter while the loop is in progress are evaluated with the specified language mode and optional environment. If you don't specify a language mode, then classic-SKILL is the default.

Note: The defining forms (such as, `define`, `defun`, `procedure`) entered at the top-level prompt are treated as “toplevel” definitions, not as local ones, even if a non-toplevel environment is supplied. (The same is true for `eval` with an explicit environment.)

Arguments

s_langMode

Must be a symbol. Valid values:

'ils Means SKILL++.

'il Means SKILL.

e_envobj

When the given *s_langMode* is for SKILL++, an optional environment object can be supplied, and the forms entered will be evaluated within the given (lexical) environment (except for the defining forms, like `define`, `defun`, and `procedure`, which will always add definitions to the top-level environment).

Value Returned

g_result

Argument passed to a call to `resume`.

Example

```
> R = toplevel( 'ils )  
ILS-<2> resume( 1 )  
1  
> R  
1
```

Cadence SKILL Development Reference

Debug Functions

Starts an interactive loop, with prompt `ILS-<2>` and immediately returns the value 1 to the outer top level.

```
ILS-<2> R = let( ( ( x 1 ) ( y 2 ) )  
toplevel( 'ils theEnvironment() )  
)
```

```
ILS-<3> x  
1  
ILS-<3> y  
2  
ILS-<3> x = 4  
4  
ILS-<3> resume( x )  
4  
ILS-<2> R  
4
```

Starts an interactive loop, with prompt `ILS-<3>`, in the environment established by the `let` expression. The `resume` function returns the current value of the local variable `x` to the outer top level, with prompt `ILS-<2>`.

Reference

[resume](#), [errset](#)

tracef

```
tracef(  
  [ { s_function | t_fileName ... | t } ]  
)  
=> g_result
```

Description

Turns on tracing of specified functions. Shows the functions called with their evaluated arguments and return values. This is an `nlambda` function.

The output port for tracing is `ptport`, which defaults to `poport`.

- If `t` is passed in, all functions are traced. However, this probably produces more information than you want and your program runs much more slowly.
- If you do not give `tracef` an argument, it returns a list of all functions being traced.
- If the argument is a string, `tracef` assumes it is a file name. `tracef` checks to see if a file was loaded after debug mode was turned on and if so, traces all functions defined in that file.
- If the symbol `debugFiles` is passed in, all functions in all files loaded since debug mode was turned on are traced.

If you want to force certain functions not to be traced even though you have turned on tracing for many or all functions, you can add the property `notrace` with a non-`nil` value on the name of the function. For example, to prevent `plus` from being traced use `putprop('plus t 'notrace )` or `plus.notrace = t`.

Cadence SKILL Development Reference

Debug Functions

Arguments

<i>s_function</i>	Any function that you want to trace.
<i>t_filename</i>	Any file containing functions that you want to trace.
<i>t</i>	Turns tracing on for all functions.

Value Returned

<i>g_result</i>	Functions or files traced.
-----------------	----------------------------

Example

```
(defun f (x) (add1 x))      ; Defines a function f.
=> f

(tracef f                  ; Turns on tracing for f.
=> (f)

f(3)
|f(3)
|f --> 4
=> 4

(tracef t                  ; Turns on tracing for all functions.
|tracef --> t
=> t

f(3)
|f(3)
||add1(3)
||add1 --> 4
|f --> 4
=> 4
```

Suppose `testfuns.il` defines functions `f1` and `f2`:

```
(defun f1 (x) x+1)
(defun f2 (y) f1(y)+1)

installDebugger            ; debug was turned on before loading a file
=> t

1> load "testfuns.il"
=> t

1> tracef("testfuns.il")
("testfuns.il")          ; tracing for all functions in the file
```


Cadence SKILL Development Reference

Debug Functions

```
1> f2 3
|f2(3)
||f1(3)
||f1 --> 4
|f2 --> 5
5
```

Reference

tracev, untrace, untracev

tracelevlimit

```
tracelevlimit(  
    [ x_depth ]  
)  
=> t
```

Description

Limits the indentation level and hence the call depth while tracing functions, arrays, or variables.

Specifying *x* traces the properties till level *x*.

Arguments

<i>x_depth</i>	Indentation level to which property(s) should be traced.
----------------	--

Value Returned

<i>t</i>	Always returns <i>t</i> .
----------	---------------------------

Example

If we define several functions, such that each function in turn calls another function, the call depth can be limited using `tracelevlimit(x_depth)`.

```
defun(func_a() var_a=1 putprop('var_aa 1 'prop_x) println("A") t)  
defun(func_b() var_b=2 putprop('var_bb 1 'prop_x) println("B") func_a() t)  
defun(func_c() var_c=3 putprop('var_cc 1 'prop_x) println("C") func_b() t)  
defun(func_d() var_d=4 putprop('var_dd 1 'prop_x) println("D") func_c() t)
```

We can set the indentation level as 2:

```
tracelevlimit(2) => t
```

So, when we call `func_d()`, the output will be:

```
|func_d()  
||putprop(var_dd 1 prop_x)  
||putprop => 1  
||println("D")  
D
```

Cadence SKILL Development Reference

Debug Functions

```
||println => nil
||func_c()
"C"
"B"
"A"
||func_c => t
|fun_d => t
t
```

tracelevunlimit

```
tracelevunlimit(  
    )  
=> t
```

Description

Turns off limiting of the the indentation level and hence the call depth while tracing functions, arrays, or variables.

Arguments

None.

Value Returned

t Always returns t.

Example

```
tracelevunlimit() => t
```

tracep

```
tracep(  
    [ { s_property...|t } ]  
)  
=> g_result
```

Description

Turns on tracing of assignments to specified properties. This is an `nlambda` function.

If `t` is passed in, all properties are traced. If no argument is given all properties being traced are returned. `tracep` prints the evaluated value assigned to the property and its previous value. The output port for tracing is `ptport`, which defaults to `poport`.

Arguments

<i>s_property</i>	Name of property(s) to be traced.
<i>t</i>	Trace all properties.

Value Returned

<i>g_result</i>	Properties successfully marked for tracing.
-----------------	---

Example

```
tracep myProp  
putprop( 'foo 5 'myProp)  
|Property myProp on foo set to 5, was nil => 5  
untracep( myProp ) => ( myProp )
```

Reference

[untracep](#)

tracev

```
tracev(  
    [ { s_variable ... | t } ]  
)  
=> g_result
```

Description

Turns on tracing of assignments to specified variables. This is an `nlambda` function.

If `t` is passed in, all variables are traced. If no argument is given all variables being traced are returned. `tracev` prints the unevaluated and evaluated value assigned to the variable. The output port for tracing is `ptport`, which defaults to `poport`.

Note: SKILL++ variables currently cannot be traced with `tracev`.

Arguments

<i>s_variable</i>	Name of the variable(s) to be traced.
<i>t</i>	Trace all variables.

Value Returned

<i>g_result</i>	Variables successfully marked for tracing.
-----------------	--

Example

```
tracev x           ; Traces the variable x.  
x = 5              ; Shows the old and new value of x.  
untracev x         ; Clears tracing for the variable x.  
tracev t           ; Traces all variable assignments.  
untracev t         ; Clears all variable tracing.
```

Reference

`tracef`, `untrace`, `untracev`

unbreakpt

```
unbreakpt(  
    [ { u_function... | t } ]  
)  
=> g_result
```

Description

Clears breakpoints. This is an `nlambda` function.

The SKILL debugger is the default break handler and is entered automatically before evaluating functions or function objects with breakpoints set. The functions `breakpt` and `unbreakpt` set and clear breakpoints on the given functions and function objects. Once you are in the break handler, you can examine the state of the program and use single stepping functions such as `step`, `next`, `stepout`, or simply `continue`.

We recommend that you turn debug mode on before using the break handler with `sstatus( debugMode t )` to change the prompts to tell you when you enter and exit the break handler. Another way to enter the break handler is to insert the `break` function directly into a SKILL function at the point desired.

Arguments

<i>u_function</i>	Function or <code>t</code> to <code>unbreakpt</code> all functions.
-------------------	---

Value Returned

<i>g_result</i>	List of functions or function objects whose breakpoints have been cleared.
-----------------	--

Example

This example sets a breakpoint, enters the break handler, continues, and finally clears the breakpoint.

```
breakpt( times )           ; Sets a breakpoint on times.  
sstatus( debugMode t )    ; Make sure debug mode is on.  
times(2 plus( 2 3 ))      ; Invokes the break handler.  
cont                      ; Continue executing.  
unbreakpt( times )        ; Clear the breakpoint.
```

This example sets a breakpoint on a function object and later clears the breakpoint.

Cadence SKILL Development Reference

Debug Functions

```
(installDebugger)           ; Make sure debug mode is on
=>t
(defun test (x) (printf "test : x == %L\n" x))
=>test
(putd 'test1 (getd 'test))   ;test and test1 are the same functions
=>funobj:test
(funcall 'breakpt (getd 'test)) ;set breakpoint on the funobject
(funobj:test)
(test1 8)
<<< Break >>> on entering test
Entering new debug toplevel due to breakpoint:
Debug 2> cont
test : x == 8
=>t
(funcall 'unbreakpt (getd 'test)) ;remove breakpoint
(funobj:test)
(test1 8)
test : x == 8
=>t
```

Reference

break, breakpt, cont, continue, next, sstatus, step, stepend, stepout

unbreakptMethod

```
unbreakptMethod(  
  [ s_name [ @before | @after ] ]  
  [ l_specializer ]  
)  
=> t
```

Description

Removes breakpoints set on the specified method.

Arguments

<i>s_name</i>	The method for which the breakpoint has to be cleared.
<i>l_specializer</i>	List of specializers of the specified method. It can include @before, @after, and @around qualifiers.

Value Returned

<i>t</i>	List of removed breakpoints.
----------	------------------------------

Example

```
unbreakptMethod(S_name)  
; removes the breakpoint on "S_name" function, behaves the same way as unbreakpt()  
unbreakptMethod(S_name nil)  
; removes the breakpoint on "generic" method of S_name.  
unbreakptMethod(S_name @after (classA t))  
; removes the breakpoint on "@after" method specialized on (classA t) of "S_name"  
unbreakptMethod(S_name (classB t))  
; removes the breakpoint on primary method specialized on (classB t) of "S_name".  
unbreakptMethod()  
; when specified without arguments, returns t
```

uncount

```
uncount(  
  [ { s_function ... | t } ]  
)  
=> g_result
```

Description

Turns off counting and returns the current count results. This is an `nlambda` function.

`count` allows you to count the number of times a function has been called. `count` and `uncount` measure function call frequency and also serve as a valuable debugging aid. Both `count` and `uncount` accept more than one function argument, or `t` for counting/uncounting all functions.

To examine the number of times a function has been counted, call the `uncount` function. A list containing the number of times each function was called, along with the function name, is returned in the form of a list of (`number functionName`) pairs, such as,

```
((20 plus) (10 times) (5 greaterp))
```

The sublists are sorted by their first elements, using `sortcar`, so the most frequently executed functions always appear at the head of the list.

Arguments

<i>s_function</i>	Turns off counting for the functions given.
<i>t</i>	Turns off counting for all functions.

Value Returned

<i>g_result</i>	List containing the number of times each function was called, along with the function name.
-----------------	---

Example

```
count( plus greaterp )      ; Counts equal and greaterp  
setq( x plus( 2 3 ))       ; Use the functions being counted.  
uncount( plus greaterp)  
=> ((1 plus)(0 greaterp))   ; Number of times each  
                             ; function was called.
```

Cadence SKILL Development Reference

Debug Functions

Reference

count

uninstallDebugger

```
uninstallDebugger(  
    )  
=> t / nil
```

Description

Uninstalls the SKILL debugger as the error handler. Turns off debug mode.

Restores the normal system error handler, which simply prints the error message and returns to the nearest toplevel. Also turns debug mode off and restores write protection on all functions.

Arguments

None.

Value Returned

t	If the debugger is successfully uninstalled.
nil	Otherwise.

Example

```
installDebugger()      ; Installs the SKILL debugger.  
4 / 0                  ; Generates an error and enters  
                        ; the SKILL debugger.  
debugQuit()           ; Exits the debugger.  
uninstallDebugger( )  ; Restores the old error handler.
```

Reference

[debugQuit](#), [installDebugger](#)

untrace

```
untrace(  
    [ { s_function | t_fileName ... | t } ]  
)  
=> g_result
```

Description

Turns tracing off for all functions specified that were traced using the `tracef` function. This is an `nlambda` function.

If the argument is a string, `untrace` assumes it is a file name and checks if the file was loaded after debug mode was turned on. If it was, it untraces all functions defined in that file.

Arguments

<i>s_function</i>	Any function that should no longer be traced.
<i>t_filename</i>	Any file containing functions that should no longer be traced.
<i>t</i>	Turns off tracing for all functions that had tracing turned on.

Value Returned

<i>g_result</i>	List of the functions or files that were untraced.
-----------------	--

Example

```
untrace( plus)
```

Turn off tracing for the `plus` function.

```
untrace( "testfns")
```

Turns off tracing for all functions in the `testfns` file assuming it was loaded after debug mode was turned on.

```
untrace( t )
```

Clears all tracing.

Reference

[tracef](#), [tracev](#), [untracev](#)

untracep

```
untracep(  
  [ { S_property...|t } ]  
)  
=> g_result
```

Description

Turns off tracing of the specified properties. This is an `nlambda` function.

Arguments

<i>S_property</i>	Name of the property(s) to be untraced.
<i>t</i>	Untrace all properties.

Value Returned

<i>g_result</i>	Properties successfully marked for untracing.
-----------------	---

Example

```
tracep myProp  
putprop( 'foo 5 'myProp)  
|Property myProp on foo set to 5, was nil  
=> 5  
untracep( myProp ) => ( myProp )
```

Reference

[tracep](#)

Cadence SKILL Development Reference

Debug Functions

untracev

```
untracev(  
    [ { s_variable ... | t } ]  
)  
=> g_result
```

Description

Turns off tracing for assignments to specified variables. This is an `nlambda` function.

Arguments

<i>s_variable</i>	Name of the variable(s) to be untraced.
<i>t</i>	Untrace all variables.

Value Returned

<i>g_result</i>	Variables successfully marked for untracing.
-----------------	--

Example

```
tracev x           ; Traces the variable x.  
x = 5              ; Shows the old and new value of x.  
untracev x         ; Clears tracing for the variable x.  
tracev t           ; Traces all variable assignments.  
untracev t         ; Clears all variable tracing.
```

Reference

[tracev](#), [tracef](#), [untrace](#)

unwatch

```
unwatch(  
    [ s_symbol...]  
)  
=> t
```

Description

Clears watchpoints set on the specified variables.

Arguments

<i>s_symbol</i>	Name of the variables for which the watchpoints need to be cleared. This argument is optional.
-----------------	--

Value Returned

<i>t</i>	Watchpoints successfully cleared.
----------	-----------------------------------

Example

```
unwatch( x ) => t
```


watch

```
watch(  
    [ s_symbol... ]  
)  
=> t
```

Description

Sets watchpoints on the specified variables.

Arguments

<i>s_symbol</i>	The variables to be watched. This argument is optional.
-----------------	---

Value Returned

<i>t</i>	Variables successfully marked for watching.
----------	---

Example

```
watch( x ) => t
```

where

```
where(  
  [ g_unevaluated ]  
  [ x_depth ]  
  [ x_skip ]  
  [ p_port ]  
)  
=> x_result
```

Description

Prints the functions on the stack and their arguments to the depth specified, or to the bottom of the stack, including the local variables and their bindings.

It is similar to `stacktrace`, but in addition to printing out the functions on the stack it also prints out the local variables and their bindings. The `where` function observes the following rules:

- When debug mode is on and the `traceArgs` status switch has been set to non-nil, prints the evaluated function arguments unless *g_unevaluated* is set to `t`.
- When debug mode is off, `where` always prints the unevaluated arguments.
- `where` skips over the number of function calls specified by *x_skip*.
- If there are no functions on the stack (you are at the very top-level, for example) `where` does not print anything and returns 0.

Prerequisites

This function is generally used inside the break or error handler.

Cadence SKILL Development Reference

Debug Functions

Arguments

<i>g_unevaluated</i>	If t, prints the unevaluated function parameters.
<i>x_depth</i>	Number of stack levels to print, defaults to all.
<i>x_skip</i>	Number of levels to skip, defaults to 1.
<i>p_port</i>	Output port. Defaults to the error port.

Value Returned

<i>x_result</i>	Number of stack frames printed.
-----------------	---------------------------------

Example

Suppose `/tmp/color.il` defines function `initColor`:

```
(defun initColor (object)
  (let ((colorList '(red green yellow)) color)
    (setq color (concat (get object 'color)))
    (if (memq color colorList)
        (printf "color %s initialized" (get object 'name)))
  )
)
```

Try this file in debugger:

```
installDebugger
=> t
1> (sstatus sourceTracing t)
                                ; Turns on sourceTracing to get line numbers
=> t

1> load "/tmp/color.il"
=> t

1> (putprop 'object1 "green" 'color)
=> "green"

1> (initColor 'object1)
*** Error in routine fprintf/sprintf:
Message: *Error* fprintf/sprintf: format spec. incompatible
with data

Debug 2> where
<<< Stack Trace >>>
errorHandler("fprintf/sprintf" 0 t nil ("*Error* fprintf/sprintf: format spec.
incompatible with data" nil))
printf("color %s initialized" get(object 'name))
at line 5 in file /tmp/color.il
if(memq(color colorList) printf("color %s initialized" get(object 'name)))
```

Cadence SKILL Development Reference

Debug Functions

```
let(((colorList '&) color) (color = concat(get(object &))) if(memq(color colorList)
printf("color %s initialized" get(object &)))
    colorList = (red green yellow)
    color = green
    object = object1
initColor('object1)
5
```

Reference

sstatus, stacktrace

whereIs

```
whereIs(  
    s_function  
)  
=> t / nil
```

Description

Prints the last file loaded in debug mode in which a function was defined, as well as the starting line number.

Prerequisites

Turn on debug mode before loading the file.

Arguments

<i>s_function</i>	Function whose file you want to locate.
-------------------	---

Value Returned

t	If the function is found.
nil	Otherwise.

Example

```
whereIs( myFunction ) => t
```

Returns `t` if the function was found and prints the file and starting line number of the `myFunction` function if it was loaded after debug mode was turned on. A sample output is

```
Function myFunction was loaded from file ~/myFunctions.il at line 126.
```

Reference

[getFunctions](#)

Cadence SKILL Development Reference

Debug Functions

Finder Functions

startFinder

```
startFinder(  
    [?funcName t_funcName]  
)  
=> t / nil
```

Description

Starts the SKILL documentation Finder utility. If the `t_funcName` argument is provided, the corresponding documentation of the function is displayed in the Finder. If a Finder window is already existing, it will be updated or a new window will be displayed.

Note: The Finder window remains open, unless specifically closed, even after the parent window has been closed.

Arguments

<code>t_funcName</code>	Name of the function to be searched in the Finder.
-------------------------	--

Value Returned

<code>t</code>	Finder is launched successfully and documentation of <code>t_funcName</code> is returned.
<code>nil</code>	Otherwise.

Example

To start the Finder utility.

```
startFinder  
=> 0
```

Cadence SKILL Development Reference

Finder Functions

To display the documentation of the function `sklint` in the Finder utility.

```
startFinder(?funcName "sklint")  
=> t
```

Tabulator Functions

skTabulate

```
skTabulate(  
    ?fileNamees g_tabulatedFileNames  
    [?reportFile t_reportFile]  
    [?showFile g_showFile]  
    [?dontResolveSymLink g_dontResolveSymLink]  
    [?customerInfo l_customerInfo]  
    [?printFiltered g_printFiltered]  
    [?ext l_ext]  
    [?infoFile g_infoFile]  
    [?defnFile g_defnFile]  
    [?recurse g_recurse]  
    [?exclude g_exclude]  
    [?recurseExclude g_recurseExclude]  
    [?sendReport g_sendReport]  
=> t
```

Description

Runs the SKILL Tabulator in batch mode.

Cadence SKILL Development Reference

Tabulator Functions

Arguments

<i>g_tabulatedFileNames</i>	The names of the files or directories to be tabulated. Should be a string or a list of strings.
<i>t_reportFile</i>	The name of the generated report file. Defaults to ". / skillTab.out".
<i>g_showFile</i>	Specifies whether to display the generated report file. Defaults to <i>nil</i> .
<i>g_dontResolveSymLink</i>	Specifies whether to resolve symbolic links. Defaults to <i>nil</i> .
<i>l_customerInfo</i>	A disembodied property list containing customer information, in the following form: <pre>'(nil customerEmail <t_email> customerName <t_name> companyLocation <t_location> companyName <t_compName> customerPhone <t_phone> custProjName <t_projName> custProjInfo <t_projInfo>)</pre>
<i>g_printFiltered</i>	Specifies whether to print user defined functions. Defaults to <i>nil</i> .
<i>l_ext</i>	List of file extensions to be tabulated. The default options are ".il", ".ile", and "cdsinit".
<i>g_infoFile</i>	The file name for the information file, which contains the names of the files that refer each function.
<i>g_defnFile</i>	The file name for the definition file, which contains the names of the files that define each function.
<i>g_recurse</i>	Specifies whether to recurse directories. Defaults to <i>t</i> .
<i>g_exclude</i>	List of files to be excluded from tabulation. Defaults to <i>nil</i> .
<i>g_recurseExclude</i>	List of recursively excluded files. Defaults to <i>nil</i> .
<i>g_sendReport</i>	Specifies whether to send the generated report to Cadence. Defaults to <i>nil</i> .

Value Returned

<i>t</i>	Always returns <i>t</i> .
----------	---------------------------

Cadence SKILL Development Reference

Tabulator Functions

Example

```
skTabulate(  
  list("./file1.il" "file2.il")  
  ?reportFile "./myTabOutput.txt"  
  ?showFile nil  
  ?dontResolveSymLink nil  
  ?customerInfo '(nil customerEmail "user@xyz.com" customerName "AN"  
    companyLocation "SJ" companyName "myCompany" customerPhone "123456"  
    custProjName "" custProjInfo "")  
  ?printFiltered t  
  ?defnFile "./myDefnFile.txt"  
  ?recurseExclude nil  
  ?infoFile "./myInfoFile.txt"  
  ?ext '("il")  
  ?sendReport t  
);
```

skTabulateSKILL

```
skTabulateSKILL(  
    )  
=> t
```

Description

Brings up the SKILL Tabulator UI form.

The SKILL Tabulator is part of the SKILL Surveyor program. Users should access the SKILL Tabulator through SKILL Surveyor, part of the Conversion tool box.

Arguments

None.

Value Returned

t Always returns t.

Example

```
skTabulateSKILL() => t
```

SKILL IDE Functions

ilgInvokeIDE

```
ilgInvokeIDE(  
    )  
=> t
```

Description

Displays the SKILL IDE main window.

Arguments

None.

Value Returned

t Always returns t.

Example

```
ilgInvokeIDE()  
=>t
```

ilgRunSKILLIDE

```
ilgRunSKILLIDE(  
    [ ?fileList lt_fileList ]  
)  
=> t / nil
```

Description

Launches SKILL IDE and opens the files listed in `lt_fileList`.

Arguments

<i>lt_fileList</i>	A list of file names (strings) that need to be opened in SKILL IDE.
--------------------	---

Value Returned

<code>t</code>	SKILL IDE was launched successfully.
<code>nil</code>	SKILL IDE could not be launched.

Example

```
ilgRunSKILLIDE(?fileList (list "demo.il"))  
=>t
```

ilgLastDir

```
ilgLastDir(  
    )  
=> t_dirPath
```

Description

Returns the directory path of the last file opened in SKILL IDE.

Arguments

None.

Value Returned

t_dirPath	Directory path of the last file opened in SKILL IDE.
-----------	--

Example

```
ilgLastDir()  
=>/home/usr1
```

ilgSetLastDir

```
ilgSetLastDir(  
    t_dirPath  
)  
=> t_path / nil
```

Description

Resets the directory path of the last file opened in SKILL IDE.

Arguments

<i>t_dirPath</i>	Path to a directory.
------------------	----------------------

Value Returned

<i>t_path</i>	The new directory path of the last file opened in SKILL IDE.
<i>nil</i>	The directory path of the last file opened in SKILL IDE was not reset.

Example

```
ilgSetLastDir("/home/usr1/test")  
=>"/home/usr1/test"
```


ilgAddRecentFiles

```
ilgAddRecentFiles(  
    l_fileList  
)  
=> t
```

Description

Adds the files listed in `l_fileList` to the SKILL IDE *File* menu.

Arguments

<code>l_fileList</code>	The list of files to be added to the SKILL IDE <i>File</i> menu.
-------------------------	--

Value Returned

<code>t</code>	The listed files were successfully added to the SKILL IDE <i>File</i> menu.
----------------	---

Example

```
ilgAddRecentFiles('("loop2.il"))  
=>t
```

Cadence SKILL Development Reference

SKILL IDE Functions
